

图论

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	iv
第一章 树上问题	1
1.1 树的直径	1
1.1.1 树上点集直径摘要	1
1.2 最近公共祖先	3
1.2.1 离线 Tarjan LCA	3
1.2.2 欧拉序与 ST 表 LCA	5
1.2.3 倍增 LCA、祖先与最大边权	8
1.2.4 两条树路径的交集	11
1.3 树链剖分	11
1.3.1 重链剖分与 LCA	11
1.4 树上启发式合并	13
1.4.1 树上启发式合并：子树众数颜色和	13
1.5 虚树	15
1.5.1 虚树：关键点压缩与路径信息	15
1.6 树分治	18
1.6.1 点分治：树上距离点对计数	18
第二章 最短路问题	21
2.1 最短路	21
2.1.1 非负权最短路	21
2.1.2 顶点开放时间的单位边最短路	23
第三章 生成树问题	24
3.1 最小树形图	24
3.1.1 有向最小树形图	24
第四章 连通性相关	26
4.1 强连通分量	26
4.1.1 Tarjan 强连通分量与缩点	26
4.1.2 函数图分解、跳转与到达	27
4.1.3 强连通分量	31
4.2 双连通分量	32
4.2.1 Tarjan 点双、边双与圆方树	32
4.2.2 属于简单奇环的全部顶点	35
4.2.3 边双缩点的大小与内部边数	37
4.2.4 无向边定向与强连通定向	37
4.3 割点和桥	38
4.3.1 由点双分解计算删点连通块数	38
4.3.2 连通无向图最少加边方案	39
4.3.3 割点与桥	40

第五章 2-SAT	42
5.1 2-SAT 与任意可行方案	42
5.2 递归染色与回滚求最小赋值	43
第六章 欧拉图	45
6.1 有向图字典序最小欧拉路	45
6.2 字典序最小单词链	47
6.3 无向图欧拉路与边方向	48
6.4 欧拉回路与指定端点定向	50
6.5 任意起终点的混合欧拉路径	52
第七章 网络流	53
7.1 最大流	53
7.1.1 最大流与方案	53
7.2 最小割	56
7.2.1 最大权闭合子图与选点方案	56
7.2.2 Gomory–Hu 最小割树	57
7.2.3 最小割树全点对查询	58
7.3 费用流	59
7.3.1 势能最小费用最大流	59
7.4 上下界网络流	62
7.4.1 上下界可行循环流	62
7.5 Stoer–Wagner 算法	63
7.5.1 无向全局最小割	63
第八章 图的匹配	65
8.1 二分图最大匹配	65
8.1.1 Hopcroft–Karp 最大匹配与最小点覆盖	65
8.2 二分图最大权匹配	67
8.2.1 矩形二分图最小权匹配	67
8.2.2 二分图最大权匹配（缺边与可选匹配）	68
8.3 一般图最大匹配	70
8.3.1 一般图最大基数匹配：带花树	70
第九章 矩阵树定理	73
9.1 带权矩阵树：无向树与内外向树形图	73
9.2 任意正模数带权矩阵树	74
第十章 配套知识与验证范围	75
10.1 非负权树的点集直径摘要	75
10.2 函数图的入环链与到达查询	76
10.3 杭州 2023 H：随机事件顺序与函数图	77
10.4 杭州 2023 G：开放时间的单位边最短路	78
10.5 SCC 模板题与速度榜口径	78
第一部分 跨类依赖	79
附录 A 数论	80
A.1 模算术简介	80

A.1.1	静态模整数	80
附录 B	线性代数	81
B.1	行列式	81
B.1.1	素数模行列式	81
B.1.2	任意正模数行列式：欧几里得消元	82
附录 C	树状数组	83
C.1	树状数组与第 k 小	83

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 树上问题

1.1 树的直径

1.1.1 树上点集直径摘要

默认空点集, $a=b=-1$ 、 $length=0$ 。insert($v,distance$) 插入顶点, 重复插入合法; merge($other,distance$) 合并两个点集摘要, 允许自合并; farthest($v,distance$) 返回 optional<pair<int,long long>> (最远顶点、距离), 空集返回 nullopt, 距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离, 点号为非负整数; 所有距离 $\leq LLONG_MAX$ 。每次插入或最远查询最多两次距离调用, 合并最多四次, 摘要只占 $O(1)$ 空间。保存 a 、 b 为点集中某对直径端点, 不保留全部点; 字段只读, 不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合, 重置用默认构造。

```
1 struct TreeDiameter
2 {
3     using ll = long long;
4     int a = -1, b = -1;
5     ll length = 0;
6
7     // distance is the metric of one fixed tree with nonnegative edge weights.
8     template <class Distance> void insert(int v, const Distance &distance)
9     {
10         assert(v >= 0);
11         if (a == -1)
12         {
13             a = v;
14             b = v;
15             return;
16         }
17         ll x = distance(a, v), y = distance(b, v);
18         if (x >= y && x > length)
19         {
20             b = v;
21             length = x;
22         }
23         else if (y > length)
24         {
25             a = v;
26             length = y;
27         }
28     }
```

合并与最远点查询（接上页同一结构体）：

```
30     template <class Distance> void merge(TreeDiameter other, const Distance &distance)
31     {
32         if (other.a == -1) return;
33         insert(other.a, distance);
34         insert(other.b, distance);
35     }
36
37     template <class Distance>
38     optional<pair<int, ll>> farthest(int v, const Distance &distance) const
39     {
40         if (a == -1) return nullopt;
41         ll x = distance(a, v), y = distance(b, v);
42         return x >= y ? make_pair(a, x) : make_pair(b, y);
43     }
44 };
```

1.2 最近公共祖先

1.2.1 离线 Tarjan LCA

构造 `OfflineLCA(n)`, `add(u,v)` 加无向树边, `add_query(u,v)` 登记查询并返回从 0 开始的编号, `run(root=1)` 后读取 `answer[id]`。输入必须是非空树, 点号 $1..n$; 允许同点、重复查询及零个查询。`run` 可重复调用或换根, 不清空查询; 追加查询后需重新 `run`, 旧答案在重算前不可按新根使用。`DFS` 与并查集查找均递归。并查集按大小合并并压缩路径, `ancestor[find(u)]` 保存已合并部分对应的当前树祖先; 不能直接拿并查集代表当 `LCA`。每个子树完成后合并到父点并更新 `ancestor`; 查询另一端已完成时, 从其集合读取祖先。总时间 $O((n+q) \alpha(n))$, 空间 $O(n+q)$ 。全部点对和根的父链对拍、50 万点链及 P3379、POJ1470 完整驱动本地验证, 在线待补。

```

1 struct OfflineLCA
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<vector<pair<int, int>>> queries;
6     vector<int> parent, size, ancestor, done, answer;
7
8     OfflineLCA(int n)
9         : n(n),
10          g(n + 1),
11          queries(n + 1),
12          parent(n + 1),
13          size(n + 1),
14          ancestor(n + 1),
15          done(n + 1)
16     {
17     }
18
19     void add(int u, int v)
20     {
21         assert(1 <= u && u <= n && 1 <= v && v <= n);
22         g[u].push_back(v);
23         g[v].push_back(u);
24     }
25
26     int add_query(int u, int v)
27     {
28         assert(1 <= u && u <= n && 1 <= v && v <= n);
29         int id = answer.size();
30         answer.push_back(0);
31         queries[u].push_back({v, id});
32         queries[v].push_back({u, id});
33         return id;
34     }

```



```

36     int find(int u)
37     {
38         if (parent[u] != u) parent[u] = find(parent[u]);
39         return parent[u];
40     }
41
42     void merge(int u, int v)
43     {
44         u = find(u);
45         v = find(v);
46         if (size[u] < size[v]) swap(u, v);
47         parent[v] = u;
48         size[u] += size[v];
49     }
50
51     void dfs(int u, int p)
52     {
53         for (int v : g[u])
54             if (v != p)
55             {
56                 dfs(v, u);
57                 merge(u, v);
58                 ancestor[find(u)] = u;
59             }
60         done[u] = 1;
61         for (auto [v, id] : queries[u])
62             if (done[v]) answer[id] = ancestor[find(v)];
63     }
64
65     // Input must be a nonempty tree. Query IDs are 0-based.
66     void run(int root = 1)
67     {
68         assert(1 <= root && root <= n);
69         iota(parent.begin(), parent.end(), 0);
70         iota(ancestor.begin(), ancestor.end(), 0);
71         fill(size.begin(), size.end(), 1);
72         fill(done.begin(), done.end(), 0);
73         fill(answer.begin(), answer.end(), 0);
74         dfs(root, 0);
75     }
76 };

```

1.2.2 欧拉序与 ST 表 LCA

构造 EulerLCA(n), add(u,v,w=1) 加非负权无向树边, build(root=1) 预处理; lca(u,v) 返回最近公共祖先, distance(u,v) 仍返回无权路径边数; weighted_distance(u,v) 返回 long long 带权距离。边权非负, 所有树上路径的权和须 $\leq \text{LLONG_MAX}$ 。输入必须为非空树, 点号 1..n; 允许同点查询和重复 build 换根。只可在 build 后查询, 加边会清除 ready 标志。递归 DFS 在进入顶点及从每个儿子返回时记录顶点, euler 长度为 2n-1, first[u] 是从 0 开始的首次出现位置; depth[root]=0。两点首次出现位置间深度最小的顶点就是 LCA, 用 ST 表保存该顶点。预处理 $O(n \log n)$ 时间和空间, 每次查询 $O(1)$ 。ST 内部下标从 0 开始; first、depth、euler、root_dist 可直接读取; root_dist 为当前根的带权距离。50 万点链、全部小树根与点对以及 P3379、POJ1330 双风格完整程序本地验证; 上述无权题在线待补; QOJ8235 的带权应用已双 AC。

```

1 struct EulerLCA
2 {
3     using ll = long long;
4     int n;
5     bool ready = false;
6     vector<vector<int>> g, st;
7     vector<vector<ll>> cost;
8     vector<ll> root_dist;
9     vector<int> first, depth, euler, lg;
10
11     EulerLCA(int n)
12         : n(n), g(n + 1), cost(n + 1), root_dist(n + 1), first(n + 1), depth(n + 1)
13     {
14     }
15
16     void add(int u, int v, ll w = 1)
17     {
18         assert(1 <= u && u <= n && 1 <= v && v <= n);
19         assert(w >= 0);
20         g[u].push_back(v);
21         g[v].push_back(u);
22         cost[u].push_back(w);
23         cost[v].push_back(w);
24         ready = false;
25     }
26
27     int shallower(int u, int v) const { return depth[u] <= depth[v] ? u : v; }

```

递归欧拉序与 RMQ (接上页同一结构体):

```

29 void dfs(int u, int parent)
30 {
31     first[u] = euler.size();
32     euler.push_back(u);
33     for (int i = 0; i < int(g[u].size()); i++)
34     {
35         int v = g[u][i];
36         if (v != parent)
37         {
38             depth[v] = depth[u] + 1;
39             root_dist[v] = root_dist[u] + cost[u][i];
40             dfs(v, u);
41             euler.push_back(u);
42         }
43     }
44 }
45
46 // Input must be a nonempty tree. Rebuilding may change the root.
47 void build(int root = 1)
48 {
49     assert(1 <= root && root <= n);
50     euler.clear();
51     depth[root] = 0;
52     root_dist[root] = 0;
53     dfs(root, 0);
54     int m = euler.size();
55     lg.assign(m + 1, 0);
56     for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
57     st.assign(lg[m] + 1, {});
58     st[0] = euler;
59     for (int k = 1; k <= lg[m]; k++)
60     {
61         st[k].resize(m - (1 << k) + 1);
62         for (int i = 0; i + (1 << k) <= m; i++)
63             st[k][i] = shallower(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
64     }
65     ready = true;
66 }

```

LCA、边数距离与带权距离（接上页同一结构体）：

```
68     int lca(int u, int v) const
69     {
70         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
71         int l = first[u], r = first[v];
72         if (l > r) swap(l, r);
73         int k = lg[r - l + 1];
74         return shallower(st[k][l], st[k][r - (1 << k) + 1]);
75     }
76
77     int distance(int u, int v) const
78     {
79         int z = lca(u, v);
80         return depth[u] + depth[v] - 2 * depth[z];
81     }
82
83     ll weighted_distance(int u, int v) const
84     {
85         int z = lca(u, v);
86         return (root_dist[u] - root_dist[z]) + (root_dist[v] - root_dist[z]);
87     }
88 };
```

1.2.3 倍增 LCA、祖先与最大边权

构造 `LiftingLCA(n)`, `add(u,v,w=1)` 加边, `build(root=1)` 预处理非空无向树, 点号 `1..n`。 `jump(u,k)` 返回 `k` 级祖先, `k` 非负, 超过根返回 `0`; `lca(u,v)` 求最近公共祖先; `distance` 返回路径边数, 不是权值和; `max_edge` 返回路径最大边权, 同点返回 `nullopt`, 其他情况返回有值的 `optional<long long>`。支持负权和 `LLONG_MIN`, `LOW` 只是最大值合并的单位元, 不表示空路径。DFS 递归, 根深度为 `0`; `up[k][u]` 与 `mx[k][u]` 是 2^k 次方长度向上段的祖先与最大边权, `0` 号节点为哨兵。可重复 `build` 换根; 修改图后必须重建。预处理 $O(n \log n)$ 时间与空间, 各查询 $O(\log n)$ 。不需要边权最值时, 可删去 `mx` 和 `max_edge` 相关部分。全小树点对/根、极值权重及 `50` 万点负权链本地验证; P3379、POJ1330 驱动只验证 LCA, 最大边权仍是本地证据。

```

1 struct LiftingLCA
2 {
3     using ll = long long;
4     static constexpr ll LOW = numeric_limits<ll>::lowest();
5     int n, levels;
6     bool ready = false;
7     vector<vector<pair<int, ll>>> g;
8     vector<vector<int>> up;
9     vector<vector<ll>> mx;
10    vector<int> depth;
11
12    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
13    {
14        assert(n > 0);
15        levels = 1;
16        while ((1LL << levels) <= n) levels++;
17        up.assign(levels, vector<int>(n + 1));
18        mx.assign(levels, vector<ll>(n + 1, LOW));
19    }
20
21    void add(int u, int v, ll w = 1)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26        ready = false;
27    }

```

```

29 void dfs(int u, int parent, ll weight)
30 {
31     up[0][u] = parent;
32     mx[0][u] = weight;
33     for (int k = 1; k < levels; k++)
34     {
35         int p = up[k - 1][u];
36         up[k][u] = up[k - 1][p];
37         mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
38     }
39     for (auto [v, w] : g[u])
40         if (v != parent)
41         {
42             depth[v] = depth[u] + 1;
43             dfs(v, u, w);
44         }
45 }
46
47 // Nonempty tree only. Node 0 means no ancestor.
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     depth[root] = 0;
52     dfs(root, 0, LOW);
53     ready = true;
54 }
55
56 int jump(int u, long long steps) const
57 {
58     assert(ready && 1 <= u && u <= n && steps >= 0);
59     if (steps > depth[u]) return 0;
60     for (int k = 0; k < levels; k++)
61         if (steps >> k & 1) u = up[k][u];
62     return u;
63 }

```

```

65     int lca(int u, int v) const
66     {
67         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
68         if (depth[u] < depth[v]) swap(u, v);
69         u = jump(u, depth[u] - depth[v]);
70         if (u == v) return u;
71         for (int k = levels - 1; k >= 0; k--)
72             if (up[k][u] != up[k][v])
73                 {
74                     u = up[k][u];
75                     v = up[k][v];
76                 }
77         return up[0][u];
78     }
79
80     int distance(int u, int v) const
81     {
82         return depth[u] + depth[v] - 2 * depth[lca(u, v)];
83     }
84
85     optional<ll> max_edge(int u, int v) const
86     {
87         assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
88         if (u == v) return nullopt;
89         if (depth[u] < depth[v]) swap(u, v);
90         ll answer = LOW;
91         int delta = depth[u] - depth[v];
92         for (int k = 0; k < levels; k++)
93             if (delta >> k & 1)
94                 {
95                     answer = max(answer, mx[k][u]);
96                     u = up[k][u];
97                 }
98         if (u == v) return answer;
99         for (int k = levels - 1; k >= 0; k--)
100             if (up[k][u] != up[k][v])
101                 {
102                     answer = max(answer, max(mx[k][u], mx[k][v]));
103                     u = up[k][u];
104                     v = up[k][v];
105                 }
106         return max(answer, max(mx[0][u], mx[0][v]));
107     }
108 };

```

1.2.4 两条树路径的交集

`path_intersection(a,b,c,d,depth,lca)` 返回路径 `a-b` 与 `c-d` 的公共路径，字段 `u`、`v` 是交集端点，`vertices` 是公共顶点数；空交返回 `{0,0,0}`，单点交的 `u=v`。点号从 1 开始，`depth` 数组与 `lca` 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 `vector` 深度表，回调接收两个点并返回 `LCA`。将 `c`、`d` 分别投影到 `a-b` 得 `p`、`q`；投影是 `lca(a,b)`、`lca(a,x)`、`lca(b,x)` 中最深者。若 `p≠q`，`c-d` 必经过 `p-q`；若 `p=q`，检验 `p` 是否在 `c-d` 上即可区分单点交与空交。不依赖顶点编号大小。常数次 `LCA` 调用、 $O(1)$ 额外空间；使用 `EulerLCA` 时为 $O(1)$ ，`HLD/倍增` 时为 $O(\log n)$ 。公共边数为 $\max(0, \text{vertices}-1)$ 。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```

1 struct PathIntersection
2 {
3     int u, v, vertices;
4 };
5
6 // Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
7 template <class Depth, class Lca>
8 PathIntersection
9 path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
10 {
11     int z = lca(a, b);
12     auto project = [&](int x)
13     {
14         int p = z;
15         for (int v : {lca(a, x), lca(b, x)})
16             if (depth[v] > depth[p]) p = v;
17         return p;
18     };
19     auto distance = [&](int u, int v)
20     {
21         return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
22     };
23     int p = project(c), q = project(d);
24     if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
25     return {p, q, int(distance(p, q) + 1)};
26 }

```

1.3 树链剖分

1.3.1 重链剖分与 LCA

`add(u,v)` 建无向树；`build(root)` 后读取 `fa`、`dep`、`siz`、`son`、`top`、`dfn`、`rk`。子树对应 $[\text{dfn}[u], \text{dfn}[u] + \text{siz}[u] - 1]$ 。`lca(u,v)` 为 $O(\log n)$ ，`path(u,v,work,edge)` 分解路径；`work(l,r)` 仅适用于可交换聚合，边权模式排除 `LCA` 位置。输入必须是连通无环树，建树 $O(n)$ ，两遍递归 `DFS`；支持改变根后重复 `build`，`dfn` 会重新编号。

```

1 struct HLD
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6
7     HLD(int n)
8         : n(n),
9           g(n + 1),
10          fa(n + 1),

```



```

11     dep(n + 1),
12     siz(n + 1),
13     son(n + 1),
14     top(n + 1),
15     dfn(n + 1),
16     rk(n + 1)
17 {
18 }
19
20 void add(int u, int v)
21 {
22     g[u].push_back(v);
23     g[v].push_back(u);
24 }
25
26 void dfs1(int u, int p)
27 {
28     fa[u] = p;
29     siz[u] = 1;
30     son[u] = 0;
31     for (int v : g[u])
32     {
33         if (v == p) continue;
34         dep[v] = dep[u] + 1;
35         dfs1(v, u);
36         siz[u] += siz[v];
37         if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
38     }
39 }
40
41 void dfs2(int u, int t)
42 {
43     top[u] = t;
44     dfn[u] = ++timer;
45     rk[timer] = u;
46     if (son[u]) dfs2(son[u], t);
47     for (int v : g[u])
48     {
49         if (v != fa[u] && v != son[u]) dfs2(v, v);
50     }
51 }
52
53 // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
54 void build(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     timer = 0;
58     dep[root] = 0;
59     dfs1(root, root);
60     dfs2(root, root);
61     assert(timer == n);
62 }
63
64 int lca(int u, int v) const
65 {
66     while (top[u] != top[v])

```

```

67     {
68         if (dep[top[u]] < dep[top[v]]) swap(u, v);
69         u = fa[top[u]];
70     }
71     return dep[u] < dep[v] ? u : v;
72 }
73
74 // Commutative operations only; edge=true excludes the LCA's position.
75 template <class F> void path(int u, int v, F work, bool edge = false) const
76 {
77     while (top[u] != top[v])
78     {
79         if (dep[top[u]] < dep[top[v]]) swap(u, v);
80         work(dfn[top[u]], dfn[u]);
81         u = fa[top[u]];
82     }
83     if (dep[u] > dep[v]) swap(u, v);
84     if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
85 }
86 };

```

1.4 树上启发式合并

1.4.1 树上启发式合并：子树众数颜色和

SubtreeColors(n), 点号 $1..n$, $\text{add}(u,v)$ 建无向树; 要求图连通且无环, $n=0$ 可空运行。run(colors, root=1) 传入长度 $n+1$ 的 int 颜色数组, colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和, max_freq[u] 为该频次; 支持负颜色与稀疏编号, 内部离散化, 和用 long long。重复 run 可更换颜色或根。重儿子信息保留, 轻儿子通过 DFS 序区间加入; 清空整棵临时子树后再重置最大值, 不尝试逐次删除时维护众数。 $O(n \log n)$ 时间、 $O(n)$ 空间, 全部 DFS 递归。insert 是内部增量统计, 可按题目修改统计量, 但相应清空规则也必须调整。

```

1 struct SubtreeColors
2 {
3     int n, best = 0;
4     long long sum = 0;
5     vector<vector<int>>> g;
6     vector<int> siz, heavy, tin, tout, order, color, value, freq;
7     vector<int> max_freq;
8     vector<long long> answer;
9
10    SubtreeColors(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
15        g[u].push_back(v);
16        g[v].push_back(u);
17    }
18
19    void prepare(int u, int p)
20    {
21        siz[u] = 1;
22        tin[u] = order.size();
23        order.push_back(u);

```

```

24     for (int v : g[u])
25     {
26         if (v == p) continue;
27         prepare(v, u);
28         siz[u] += siz[v];
29         if (siz[v] > siz[heavy[u]]) heavy[u] = v;
30     }
31     tout[u] = (int)order.size() - 1;
32 }
33
34 void insert(int u)
35 {
36     int c = color[u];
37     freq[c]++;
38     if (freq[c] > best)
39     {
40         best = freq[c];
41         sum = value[c];
42     }
43     else if (freq[c] == best)
44         sum += value[c];
45 }
46
47 void solve(int u, int p, bool keep)
48 {
49     for (int v : g[u])
50     {
51         if (v != p && v != heavy[u]) solve(v, u, false);
52     }
53     if (heavy[u]) solve(heavy[u], u, true);
54     for (int v : g[u])
55     {
56         if (v == p || v == heavy[u]) continue;
57         for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
58     }
59     insert(u);
60     max_freq[u] = best;
61     answer[u] = sum;
62     if (!keep)
63     {
64         for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
65         best = 0;
66         sum = 0;
67     }
68 }
69
70 // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
71 void run(const vector<int> &colors, int root = 1)
72 {
73     assert((int)colors.size() == n + 1);
74     siz.assign(n + 1, 0);
75     heavy = siz;
76     tin = siz;
77     tout = siz;
78     color = siz;
79     max_freq = siz;

```

```

80     answer.assign(n + 1, 0);
81     order.clear();
82     value.assign(colors.begin() + 1, colors.end());
83     sort(value.begin(), value.end());
84     value.erase(unique(value.begin(), value.end()), value.end());
85     freq.assign(value.size(), 0);
86     best = 0;
87     sum = 0;
88     if (!n) return;
89     assert(1 <= root && root <= n);
90     for (int u = 1; u <= n; u++)
91         color[u] =
92             lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
93     prepare(root, 0);
94     solve(root, 0, false);
95 }
96 };

```

1.5 虚树

1.5.1 虚树：关键点压缩与路径信息

VirtualTree(n), $n \geq 1$, 点号 $1..n$; add($u,v,w=1$) 建非负权连通树, 无环和重边。prepare($root=1$) 递归预处理, 可换根重建; 根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA, 返回 vertices 和 edges; 全部编号沿用原树, vertices 按 DFS 序排列, 非空时首项即虚树根。edges 为 (from,to,distance,minimum), 分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果, 单点返回单节点; 原树根不会强制加入, 需要时由调用者加入 keys。边按后代 DFS 序排列, 可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表; 每次仅返回 $O(k)$ 结果。预处理 $O(n \log n)$ 时间空间, 单次 $O(k \log k + k \log n)$ 。lca、ancestor 也可直接使用, 要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法, 原树 DFS 保持递归。

```

1 struct VirtualTree
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll distance, minimum;
9     };
10
11     struct Result
12     {
13         vector<int> vertices;
14         vector<Edge> edges;
15     };
16
17     int n, log = 1, timer = 0;
18     vector<vector<pair<int, ll>>> g;
19     vector<vector<int>> up;
20     vector<vector<ll>> mn;
21     vector<int> depth, tin, tout;
22     vector<ll> dist;
23
24     VirtualTree(int n) : n(n), g(n + 1) {}

```

```

25
26 void add(int u, int v, ll w = 1)
27 {
28     assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
29     g[u].push_back({v, w});
30     g[v].push_back({u, w});
31 }
32
33 void dfs(int u, int p)
34 {
35     tin[u] = ++timer;
36     up[0][u] = p;
37     for (int j = 1; j < log; j++)
38     {
39         up[j][u] = up[j - 1][up[j - 1][u]];
40         mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
41     }
42     for (auto [v, w] : g[u])
43     {
44         if (v == p) continue;
45         assert(dist[u] <= numeric_limits<ll>::max() - w);
46         depth[v] = depth[u] + 1;
47         dist[v] = dist[u] + w;
48         mn[0][v] = w;
49         dfs(v, u);
50     }
51     tout[u] = timer;
52 }
53
54 void prepare(int root = 1)
55 {
56     assert(1 <= root && root <= n);
57     log = 1;
58     while ((1LL << log) <= n) log++;
59     up.assign(log, vector<int>(n + 1));
60     mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
61     depth.assign(n + 1, 0);
62     tin = depth;
63     tout = depth;
64     dist.assign(n + 1, 0);
65     timer = 0;
66     dfs(root, root);
67 }
68
69 bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }
70
71 int lca(int u, int v) const
72 {
73     if (ancestor(u, v)) return u;
74     if (ancestor(v, u)) return v;
75     for (int j = log - 1; j >= 0; j--)
76     {
77         if (!ancestor(up[j][u], v)) u = up[j][u];
78     }
79     return up[0][u];
80 }

```

```

81
82 Edge path(int u, int v) const
83 {
84     assert(u != v && ancestor(u, v));
85     ll minimum = numeric_limits<ll>::max();
86     int x = v, steps = depth[v] - depth[u];
87     for (int j = 0; j < log; j++)
88     {
89         if (steps >> j & 1)
90         {
91             minimum = min(minimum, mn[j][x]);
92             x = up[j][x];
93         }
94     }
95     return {u, v, dist[v] - dist[u], minimum};
96 }
97
98 // Uses original vertex IDs. Empty input gives an empty result.
99 Result build(vector<int> keys) const
100 {
101     if (keys.empty()) return {};
102     for (int u : keys) assert(1 <= u && u <= n);
103     auto cmp = [&](int u, int v)
104     {
105         return tin[u] < tin[v];
106     };
107     sort(keys.begin(), keys.end(), cmp);
108     keys.erase(unique(keys.begin(), keys.end()), keys.end());
109     int k = keys.size();
110     for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
111     sort(keys.begin(), keys.end(), cmp);
112     keys.erase(unique(keys.begin(), keys.end()), keys.end());
113     Result result{keys, {}};
114     vector<int> st;
115     for (int u : keys)
116     {
117         while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
118         if (!st.empty()) result.edges.push_back(path(st.back(), u));
119         st.push_back(u);
120     }
121     return result;
122 }
123 };

```

1.6 树分治

1.6.1 点分治：树上距离点对计数

CentroidPairs(n), 点号 $1..n$; add(u,v,w) 建树, 允许森林但禁止环、重边、自环, 边权非负; 所有简单路径长度须能用 long long 表示。build() 预处理, 可重复调用; 改变边后须重建。count_leq(k) 统计距离 $\leq k$ 的无序不同顶点对, count_exact(k) 统计距离恰为 k 的点对; 负 k 返回 0, 零距离只统计不同顶点, 跨连通块不配对。parent[c] 是点分树父节点, 根为 0。all[c] 为该层分治块到重心的距离, 含重心距离 0; part[c] 为去掉重心后各子块的距离。预处理 $O(n \log^2 n)$, 空间 $O(n \log n)$, 每次计数 $O(n \log n)$; 固定树上多次查询可以复用排序结果。DFS 递归, 距离和比较采用减法避免溢出。不是动态点分树更新模板。

```

1 struct CentroidPairs
2 {
3     using ll = long long;
4     int n;
5     vector<vector<pair<int, ll>>> g;
6     vector<int> siz, removed, parent;
7     vector<vector<ll>> all;
8     vector<vector<vector<ll>>> part;
9
10    CentroidPairs(int n)
11        : n(n),
12          g(n + 1),
13          siz(n + 1),
14          removed(n + 1),
15          parent(n + 1),
16          all(n + 1),
17          part(n + 1)
18    {
19    }
20
21    void add(int u, int v, ll w)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
24        g[u].push_back({v, w});
25        g[v].push_back({u, w});
26    }
27
28    void size_dfs(int u, int p)
29    {
30        siz[u] = 1;
31        for (auto [v, w] : g[u])
32        {
33            if (v == p || removed[v]) continue;
34            size_dfs(v, u);
35            siz[u] += siz[v];
36        }
37    }
38
39    int centroid(int u, int p, int total)
40    {
41        for (auto [v, w] : g[u])
42        {
43            if (v != p && !removed[v] && siz[v] > total / 2)
44                return centroid(v, u, total);
45        }

```

```

46     return u;
47 }
48
49 void collect(int u, int p, ll d, vector<ll> &out)
50 {
51     out.push_back(d);
52     for (auto [v, w] : g[u])
53     {
54         if (v == p || removed[v]) continue;
55         assert(d <= numeric_limits<ll>::max() - w);
56         collect(v, u, d + w, out);
57     }
58 }
59
60 void decompose(int entry, int p)
61 {
62     size_dfs(entry, 0);
63     int c = centroid(entry, 0, siz[entry]);
64     removed[c] = 1;
65     parent[c] = p;
66     all[c] = {0};
67     for (auto [v, w] : g[c])
68     {
69         if (removed[v]) continue;
70         part[c].push_back({});
71         auto &d = part[c].back();
72         collect(v, c, w, d);
73         sort(d.begin(), d.end());
74         all[c].insert(all[c].end(), d.begin(), d.end());
75     }
76     sort(all[c].begin(), all[c].end());
77     for (auto [v, w] : g[c])
78     {
79         if (!removed[v]) decompose(v, c);
80     }
81 }
82
83 // Input must be a forest. Rebuild after changing edges.
84 void build()
85 {
86     fill(removed.begin(), removed.end(), 0);
87     for (int u = 1; u <= n; u++)
88     {
89         all[u].clear();
90         part[u].clear();
91     }
92     for (int u = 1; u <= n; u++)
93     {
94         if (!removed[u]) decompose(u, 0);
95     }
96 }
97
98 ll pairs(const vector<ll> &d, ll k) const
99 {
100     int l = 0, r = (int)d.size() - 1;
101     ll answer = 0;

```



```

102     while (l < r)
103     {
104         if (d[l] <= k - d[r])
105         {
106             answer += r - l;
107             l++;
108         }
109         else
110             r--;
111     }
112     return answer;
113 }
114
115 // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
116 ll count_leq(ll k) const
117 {
118     if (k < 0) return 0;
119     ll answer = 0;
120     for (int c = 1; c <= n; c++)
121     {
122         answer += pairs(all[c], k);
123         for (const auto &d : part[c]) answer -= pairs(d, k);
124     }
125     return answer;
126 }
127
128 ll count_exact(ll k) const
129 {
130     if (k < 0) return 0;
131     return count_leq(k) - (k ? count_leq(k - 1) : 0);
132 }
133 };

```

第二章 最短路问题

2.1 最短路

2.1.1 非负权最短路

$\text{add}(u,v,w)$ 、 $\text{run}(s)$ ， dis 为距离， pre 为前驱， $\text{path}(t)$ 返回点序列。不可达用 LLONG_MAX ；要求所有有限最短距离严格小于该哨兵， $w \geq 0$ 。 $O((V+E)\log V)$ 。

```
1 struct Dijkstra
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9
10    Dijkstra(int n) : n(n), g(n + 1) {}
11
12    void add(int u, int v, ll w)
13    {
14        assert(w >= 0);
15        g[u].push_back({v, w});
16    }
17
18    void run(int s)
19    {
20        dis.assign(n + 1, inf);
21        pre.assign(n + 1, -1);
22        dis[s] = 0;
23        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
24        q.push({0, s});
25        while (!q.empty())
26        {
27            auto [d, u] = q.top();
28            q.pop();
29            if (d != dis[u]) continue;
30            for (auto [v, w] : g[u])
31                if (w < inf - d && d + w < dis[v])
32                {
33                    dis[v] = d + w;
34                    pre[v] = u;
35                    q.push({dis[v], v});
36                }
37        }
38    }
```

```
39
40     vector<int> path(int t) const
41     {
42         if (dis[t] == inf) return {};
43         vector<int> ans;
44         for (; t != -1; t = pre[t]) ans.push_back(t);
45         reverse(ans.begin(), ans.end());
46         return ans;
47     }
48 };
```

2.1.2 顶点开放时间的单位边最短路

`release_bfs(open,source,neighbors)` 返回各点最早到达时间,不可达为 -1; 点号 $0..n-1$, $n>0$, `open[v]` 为非负整数, 源点初始时间为 `open[source]`。允许原地等待, 每条边耗时恰为 1, `neighbors(u,visit)` 枚举所有出邻点并调用 `visit(v)`, 可直接枚举网格而不建边。令 $R=\max(\text{open})$, 要求 $R+n<\text{INT_MAX}$ 且内存可承受; 时间 $O(n+m+R)$, 空间 $O(n+R)$ 。第一次发现即可定距, 只适用于单位边和固定开放时间; 不能用于一般带权边、关闭时间或周期可用边。

```

1 template <class Neighbors>
2 vector<int> release_bfs(const vector<int> &open, int source, Neighbors neighbors)
3 {
4     int n = open.size();
5     assert(0 <= source && source < n);
6     int last = *max_element(open.begin(), open.end());
7     assert(last >= 0 && last < INT_MAX - n);
8     vector<vector<int>> pending(last + 1);
9     vector<int> dist(n, -1), queue;
10    queue.reserve(n);
11    dist[source] = open[source];
12    queue.push_back(source);
13    int head = 0, time = 0, waiting = 0;
14    while (head < int(queue.size()) || waiting)
15    {
16        int limit = head < int(queue.size()) ? dist[queue[head]] : INT_MAX;
17        while (time <= last && time <= limit && pending[time].empty()) time++;
18        int u;
19        if (time <= last && time <= limit)
20        {
21            u = pending[time].back();
22            pending[time].pop_back();
23            waiting--;
24        }
25        else
26            u = queue[head++];
27        neighbors(u,
28                [&](int v)
29                {
30                    if (dist[v] != -1) return;
31                    dist[v] = max(dist[u] + 1, open[v]);
32                    if (dist[v] == dist[u] + 1)
33                        queue.push_back(v);
34                    else
35                    {
36                        pending[open[v]].push_back(v);
37                        waiting++;
38                    }
39                });
40    }
41    return dist;
42 }
```

第三章 生成树问题

3.1 最小树形图

3.1.1 有向最小树形图

`solve(n,root,edges)`, 点号 $0..n-1$, $n \geq 1$, $0 \leq \text{root} < n$; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边和自环 (自环不选), $n=1$ 的费用为 0; 费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^{120} , 类型为 `int128`。每个非根点选最小入边; 若成环则缩点, 跨环入边减去原终点已选入边费用, 以计算替代代价。 $O(VE)$, `edges` 按值传入, 原图保持不变。仅返回费用, 不恢复边方案。双版本已做所有根の入边枚举与 800 点连续缩环核验, 暂无专门 OJ AC。

```
1 struct Arborescence
2 {
3     using I = __int128_t;
4
5     struct Edge
6     {
7         int u, v;
8         I w;
9     };
10
11     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
12     static optional<I> solve(int n, int root, vector<Edge> e)
13     {
14         const I inf = I(1) << 120;
15         I ans = 0;
16         while (true)
17         {
18             vector<I> in(n, inf);
19             vector<int> pre(n, -1), id(n, -1), vis(n, -1);
20             for (auto a : e)
21                 if (a.u != a.v && a.w < in[a.v])
22                 {
23                     in[a.v] = a.w;
24                     pre[a.v] = a.u;
25                 }
26             in[root] = 0;
27             for (I x : in)
28             {
29                 if (x == inf) return nullopt;
30                 ans += x;
31             }

```

判环与缩点（接上页同一函数）：

```
32     int cnt = 0;
33     for (int i = 0; i < n; i++)
34     {
35         int u = i;
36         while (vis[u] != i && id[u] == -1 && u != root)
37         {
38             vis[u] = i;
39             u = pre[u];
40         }
41         if (u != root && id[u] == -1)
42         {
43             id[u] = cnt;
44             for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
45             ++cnt;
46         }
47     }
48     if (!cnt) return ans;
49     for (int i = 0; i < n; i++)
50         if (id[i] == -1) id[i] = cnt++;
51     for (auto &a : e)
52     {
53         int v = a.v;
54         a.u = id[a.u];
55         a.v = id[a.v];
56         if (a.u != a.v) a.w -= in[v];
57     }
58     root = id[root];
59     n = cnt;
60 }
61 }
62 };
```

第四章 连通性相关

4.1 强连通分量

4.1.1 Tarjan 强连通分量与缩点

TarjanSCC(n), 点号 1..n; add(u,v) 加有向边, run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号, 跨分量边总从大编号指向小编号, 可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表, 保留重边; 若 DP 需要对不同邻点去重, 调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 $O(n+m)$, 递归 DFS 另需 $O(n)$ 调用栈; 本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
1 struct TarjanSCC
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<int>> g;
5     vector<int> dfn, low, bel, st;
6
7     TarjanSCC(int n) : n(n), g(n + 1) {}
8
9     void add(int u, int v)
10    {
11        assert(1 <= u && u <= n && 1 <= v && v <= n);
12        g[u].push_back(v);
13    }
14
15    void dfs(int u)
16    {
17        dfn[u] = low[u] = ++timer;
18        st.push_back(u);
19        for (int v : g[u])
20        {
21            if (!dfn[v])
22            {
23                dfs(v);
24                low[u] = min(low[u], low[v]);
25            }
26            else if (!bel[v])
27                low[u] = min(low[u], dfn[v]);
28        }
29        if (low[u] == dfn[u])
30        {
31            ++cnt;
32            while (true)
33            {
34                int v = st.back();
35                st.pop_back();
36                bel[v] = cnt;
37                if (v == u) break;
38            }
39        }
40    }
41 }
```

```

42 // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
43 void run()
44 {
45     timer = 0;
46     cnt = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     bel = dfn;
50     st.clear();
51     for (int u = 1; u <= n; u++)
52     {
53         if (!dfn[u]) dfs(u);
54     }
55 }
56
57 // Retains parallel edges. Use after run().
58 vector<vector<int>> dag() const
59 {
60     vector<vector<int>> result(cnt + 1);
61     for (int u = 1; u <= n; u++)
62     {
63         for (int v : g[u])
64         {
65             if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
66         }
67     }
68     return result;
69 }
70 };

```

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

4.1.2 函数图分解、跳转与到达

FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n ≤ INT_MAX，实际规模须满足内存。

```

1 struct FunctionalGraph
2 {
3     using U = unsigned long long;
4     int n;
5     vector<int> component, position, entry, depth, order;
6     vector<vector<int>> cycles, up;
7
8     explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }
9
10    void build(vector<int> to)
11    {
12        assert(to.size() <= INT_MAX);
13        n = to.size();

```



```
14     vector<int> degree(n);
15     for (int v : to)
16     {
17         assert(0 <= v && v < n);
18         degree[v]++;
19     }
20     order.clear();
21     for (int u = 0; u < n; u++)
22         if (degree[u] == 0) order.push_back(u);
23     for (int i = 0; i < int(order.size()); i++)
24     {
25         int v = to[order[i]];
26         if (--degree[v] == 0) order.push_back(v);
27     }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```

28     component.assign(n, -1);
29     position.assign(n, -1);
30     entry.resize(n);
31     depth.assign(n, 0);
32     cycles.clear();
33     for (int u = 0; u < n; u++)
34         if (degree[u] && component[u] == -1)
35         {
36             vector<int> cycle;
37             int v = u, id = cycles.size();
38             do
39             {
40                 component[v] = id;
41                 position[v] = cycle.size();
42                 entry[v] = v;
43                 cycle.push_back(v);
44                 v = to[v];
45             } while (v != u);
46             cycles.push_back(move(cycle));
47         }
48     for (int i = int(order.size()) - 1; i >= 0; i--)
49     {
50         int u = order[i], v = to[u];
51         component[u] = component[v];
52         entry[u] = entry[v];
53         depth[u] = depth[v] + 1;
54     }
55     up.assign(1, to);
56     for (int k = 1; (1ULL << k) <= U(n); k++)
57     {
58         vector<int> row(n);
59         for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
60         up.push_back(move(row));
61     }
62 }
```

跳转与最少步数接口（接上页同一结构体）:

```

64     int advance(int u, U k) const
65     {
66         assert(0 <= u && u < n);
67         if (k >= U(depth[u]))
68         {
69             k -= depth[u];
70             u = entry[u];
71             const auto &cycle = cycles[component[u]];
72             return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
73         }
74         for (int bit = 0; k; bit++, k >>= 1)
75             if (k & 1) u = up[bit][u];
76         return u;
77     }
78
79     int steps(int u, int v) const
80     {
81         assert(0 <= u && u < n && 0 <= v && v < n);
82         if (component[u] != component[v] || depth[u] < depth[v]) return -1;
83         if (depth[v] > 0)
84         {
85             int d = depth[u] - depth[v];
86             return advance(u, d) == v ? d : -1;
87         }
88         int offset = position[v] - position[entry[u]];
89         if (offset < 0) offset += int(cycles[component[u]].size());
90         return depth[u] + offset;
91     }
92 };

```

4.1.3 强连通分量

$\text{add}(u, v)$ 、 $\text{run}()$ 、 $\text{bel}[1..n]$ 为分量编号， cnt 为数量；分量编号按缩点图拓扑序递增。递归 Kosaraju，按用户要求保留 DFS 写法； $O(V+E)$ ，同时存正反图。

```

1 struct SCC
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
6
7     SCC(int n) : n(n), g(n + 1), rg(n + 1) {}
8
9     void add(int u, int v)
10    {
11        g[u].push_back(v);
12        rg[v].push_back(u);
13    }
14
15    void dfs1(int u, vector<int> &vis, vector<int> &order)
16    {
17        vis[u] = 1;
18        for (int v : g[u])
19        {
20            if (!vis[v]) dfs1(v, vis, order);
21        }
22        order.push_back(u);
23    }
24
25    void dfs2(int u)
26    {
27        bel[u] = cnt;
28        for (int v : rg[u])
29        {
30            if (!bel[v]) dfs2(v);
31        }
32    }
33
34    // Recursive Kosaraju; component IDs are in topological order, 1..cnt.
35    void run()
36    {
37        vector<int> vis(n + 1), order;
38        cnt = 0;
39        bel.assign(n + 1, 0);
40        for (int s = 1; s <= n; s++)
41        {
42            if (!vis[s]) dfs1(s, vis, order);
43        }
44        reverse(order.begin(), order.end());
45        for (int s : order)

```

```

46     {
47         if (!bel[s])
48         {
49             ++cnt;
50             dfs2(s);
51         }
52     }
53 }
54 };

```

4.2 双连通分量

4.2.1 Tarjan 点双、边双与圆方树

Biconnected(n), 点号 1..n; add(u,v) 返回原边的 0-based 编号, run() 从头计算。blocks 为点双的顶点列表: 桥的两端构成一个块, 孤立点独立成块; 同一个割点可出现在多个块。cut[u] 标记割点, bridge[id] 标记桥; bel[u] 是 1..cnt 的边双编号。block_forest() 返回圆方森林, 原点 1..n, blocks[i] 的方点编号 n+i+1; 孤立点也连接自己的单点方块。bridge_forest() 返回边双缩点森林, 邻接项为 (相邻分量, 原桥编号)。重边按边号处理; 自环保留原边编号但不参与 DFS, 不是桥, 不额外产生点双。全程 $O(n+m)$, 递归 DFS, 分量栈不替代递归调用。两种森林必须在 run 后获取。

```

1 struct Biconnected
2 {
3     int n, timer = 0, cnt = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<vector<int>> blocks;
7     vector<int> dfn, low, cut, bridge, bel, st;
8
9     Biconnected(int n) : n(n), g(n + 1) {}
10
11     int add(int u, int v)
12     {
13         assert(1 <= u && u <= n && 1 <= v && v <= n);
14         int id = edges.size();
15         edges.push_back({u, v});
16         if (u != v)
17         {
18             g[u].push_back({v, id});
19             g[v].push_back({u, id});
20         }
21         return id;
22     }
23
24     void dfs(int u, int parent_edge)
25     {
26         dfn[u] = low[u] = ++timer;
27         st.push_back(u);
28         int children = 0;
29         for (auto [v, id] : g[u])
30         {
31             if (id == parent_edge) continue;
32             if (!dfn[v])
33             {

```

```

34         children++;
35         dfs(v, id);
36         low[u] = min(low[u], low[v]);
37         if (low[v] > dfn[u]) bridge[id] = 1;
38         if (low[v] >= dfn[u])
39         {
40             if (parent_edge != -1 || children > 1) cut[u] = 1;
41             blocks.push_back({u});
42             while (true)
43             {
44                 int x = st.back();
45                 st.pop_back();
46                 blocks.back().push_back(x);
47                 if (x == v) break;
48             }
49         }
50     }
51     else
52         low[u] = min(low[u], dfn[v]);
53 }
54 if (parent_edge == -1 && children == 0) blocks.push_back({u});
55 }
56
57 void paint(int u)
58 {
59     bel[u] = cnt;
60     for (auto [v, id] : g[u])
61     {
62         if (!bridge[id] && !bel[v]) paint(v);
63     }
64 }
65
66 void run()
67 {
68     timer = 0;
69     cnt = 0;
70     dfn.assign(n + 1, 0);
71     low = dfn;
72     cut = dfn;
73     bel = dfn;
74     bridge.assign(edges.size(), 0);
75     blocks.clear();
76     st.clear();
77     for (int u = 1; u <= n; u++)
78     {
79         if (!dfn[u])
80         {
81             dfs(u, -1);
82             st.pop_back();
83         }
84     }
85     for (int u = 1; u <= n; u++)
86     {
87         if (bel[u]) continue;
88         ++cnt;
89         paint(u);

```

```

90     }
91 }
92
93 // Original vertices 1..n; block i has node n+i+1. Includes isolated vertices.
94 vector<vector<int>> block_forest() const
95 {
96     vector<vector<int>> tree(n + blocks.size() + 1);
97     for (int i = 0; i < (int)blocks.size(); i++)
98     {
99         int b = n + i + 1;
100         for (int u : blocks[i])
101         {
102             tree[u].push_back(b);
103             tree[b].push_back(u);
104         }
105     }
106     return tree;
107 }
108
109 // Edge-connected component IDs 1..cnt; forest edges carry original edge IDs.
110 vector<vector<pair<int, int>>> bridge_forest() const
111 {
112     vector<vector<pair<int, int>>> tree(cnt + 1);
113     for (int id = 0; id < (int)edges.size(); id++)
114     {
115         if (!bridge[id]) continue;
116         auto [u, v] = edges[id];
117         u = bel[u];
118         v = bel[v];
119         tree[u].push_back({v, id});
120         tree[v].push_back({u, id});
121     }
122     return tree;
123 }
124 };

```

4.2.2 属于简单奇环的全部顶点

先对 Biconnected 调用 run，再调用 odd_cycle_vertices(graph)，返回 answer[1..n]，值为 1 表示该点属于某个长度至少为 3 的简单奇环，answer[0]=0。允许重边、断开图及空图；自环不计入这里的奇环。某点双非二分当且仅当其中存在奇环，此时该点双每个顶点都属于某个奇环；答案为所有非二分点双的顶点并集。通过圆方森林把每条非自环边分到唯一点双：两端中森林深度较大者的父方点即该块，同深时二者父方点相同。随后只对块内边染色，避免在每个块中重复扫描割点的全部邻边。两次 DFS 均递归，分解后额外 $O(n+m)$ 时间和空间。可重排 blocks，不能改变分解内容。六点以内所有简单图枚举奇环对拍，另有重边/自环、长链、星形和大奇环测试；POJ2942 补图驱动本地验证，在线待补。依赖 biconnected.hpp。

```

1 // After run(). Simple odd cycles have length >= 3; self-loops are ignored.
2 inline vector<int> odd_cycle_vertices(const Biconnected &graph)
3 {
4     int n = graph.n;
5     auto forest = graph.block_forest();
6     vector<int> parent(forest.size(), depth(forest.size()));
7     auto root_tree = [&](auto &&self, int u, int p) -> void
8     {
9         parent[u] = p;
10        for (int v : forest[u])
11            if (v != p)
12            {
13                depth[v] = depth[u] + 1;
14                self(self, v, u);
15            }
16    };
17    for (int u = 1; u <= n; u++)
18        if (!depth[u])
19        {
20            depth[u] = 1;
21            root_tree(root_tree, u, 0);
22        }

```



```

23     vector<vector<pair<int, int>>> edges(graph.blocks.size());
24     for (auto [u, v] : graph.edges)
25         if (u != v)
26         {
27             int block = parent[depth[u] >= depth[v] ? u : v] - n - 1;
28             assert(0 <= block && block < (int)edges.size());
29             edges[block].push_back({u, v});
30         }
31     vector<vector<int>> g(n + 1);
32     vector<int> color(n + 1), answer(n + 1);
33     for (int block = 0; block < (int)edges.size(); block++)
34     {
35         for (int u : graph.blocks[block])
36         {
37             g[u].clear();
38             color[u] = 0;
39         }
40         for (auto [u, v] : edges[block])
41         {
42             g[u].push_back(v);
43             g[v].push_back(u);
44         }
45         bool bipartite = true;
46         auto paint = [&](auto &&self, int u) -> void
47         {
48             for (int v : g[u])
49                 if (!color[v])
50                 {
51                     color[v] = -color[u];
52                     self(self, v);
53                 }
54             else if (color[v] == color[u])
55                 bipartite = false;
56         };
57         int root = graph.blocks[block][0];
58         color[root] = 1;
59         paint(paint, root);
60         if (!bipartite)
61             for (int u : graph.blocks[block]) answer[u] = 1;
62     }
63     return answer;
64 }

```

4.2.3 边双缩点的大小与内部边数

在 `Biconnected.run()` 后构造 `EdgeCompression(graph)`。n 为边双数量；size[c] 为第 c 块的原顶点数，inside[c] 为块内逻辑无向边数，分量号 1..n，0 号槽忽略。edges 每项为 {块 u, 块 v, 原边 id}，只含跨块的桥，每条桥出现一次，按原边号递增；是无向森林，不承诺端点为父子方向。重边逐条计数，自环在所属块内部计一次，不能用邻接表度数直接计边而漏掉这些约定。空图返回 n=0、长度 1 的零计数数组和空边表。时间 $O(N+M)$ ，输出空间 $O(N+M)$ ，N/M 指原图规模。

```

1 struct EdgeCompression
2 {
3     int n;
4     vector<int> size, inside;
5     vector<array<int, 3>> edges;
6
7     EdgeCompression(const Biconnected &graph) : n(graph.cnt), size(n + 1), inside(n + 1)
8     {
9         for (int u = 1; u <= graph.n; u++) size[graph.bel[u]]++;
10        for (int id = 0; id < (int)graph.edges.size(); id++)
11        {
12            auto [u, v] = graph.edges[id];
13            u = graph.bel[u];
14            v = graph.bel[v];
15            if (u == v)
16                inside[u]++;
17            else
18                edges.push_back({u, v, id});
19        }
20    }
21 };

```

4.2.4 无向边定向与强连通定向

`orient_edges(graph)` 接受 Lowlink 对象。返回长度等于逻辑边数的方向数组，第 id 项为该边定向后的 {起点, 终点}，顶点 1-based、边 id 为 0-based。每条无向边仅处理一次：DFS 树边向下，非树边指向祖先，自环保留 {u,u}，重边不合并。支持非连通图；每个边双块内定向后强连通，桥只给出某个方向，不保证跨桥互相可达。若要求整个非空图强连通，需另外 run/Run 检查 components==1 且无桥；不满足则不存在这样的方向。该函数只读取邻接表与边数，不依赖已有 DFS 状态，不修改原图，调用时无须先 run。复杂度 $O(N+M)$ 时间、 $O(N+M)$ 额外空间，DFS 保持递归。

```

1 // Tree edges go down; other edges go to ancestors. Handles each logical edge once.
2 inline vector<pair<int, int>> orient_edges(const Lowlink &graph)
3 {
4     vector<pair<int, int>> direction(graph.edges);
5     vector<int> seen(graph.n + 1);
6     auto dfs = [&](auto &&self, int u) -> void
7     {
8         seen[u] = 1;
9         for (auto [v, id] : graph.g[u])
10        {

```

```

11         if (direction[id].first) continue;
12         direction[id] = {u, v};
13         if (!seen[v]) self(self, v);
14     }
15 };
16 for (int u = 1; u <= graph.n; u++)
17     if (!seen[u]) dfs(dfs, u);
18 return direction;
19 }

```

4.3 割点和桥

4.3.1 由点双分解计算删点连通块数

在 `Biconnected.run()` 后调用 `removal_components(graph)`, 返回 (原连通块数 `before`, 删点结果 `after`)。 `after[u]` 为独立删除原顶点 `u` 后剩余连通块数, 点号 `1..n`; `after[0]=0` 不参与计算。 `before` 等于边双数量减去桥数量; `after[u]` 等于 `before-1` 加包含 `u` 的非单点点双数量。单点块必须跳过, 因此孤立点删去后会少一个连通块。空图返回 `(0,{0})`。不修改原分解, $O(n+m)$ 时间、 $O(n)$ 额外空间。若只需要割点、桥和删点增量, 直接用更短的 `Lowlink/Low_Link`; 已经需要点双分解时可用本函数, 避免再次 DFS。

```

1 // Call after run(). Each vertex is deleted independently from the original graph.
2 inline pair<int, vector<int>> removal_components(const Biconnected &graph)
3 {
4     int before = graph.cnt - count(graph.bridge.begin(), graph.bridge.end(), 1);
5     vector<int> after(graph.n + 1, before - 1);
6     after[0] = 0;
7     for (const auto &block : graph.blocks)
8         if (block.size() > 1)
9             for (int u : block) after[u]++;
10    return {before, after};
11 }

```

4.3.2 连通无向图最少加边方案

先对非空连通无向图调用 `Biconnected.run()`，再调用 `bridge_augmentation(graph)`。返回原图点号构成的新增边列表，不修改原图。允许新增重边，不产生自环；不适用于非连通图。已无桥时返回空列表。桥树有 L 个叶子时最少加 $(L+1)/2$ 条边（整数除法）。递归 DFS 顺序收集叶子，奇数时复制首叶，前后半段配对。每条桥一侧的叶子是循环连续区间，半周配对必跨过该割，故所有桥被消除；每个叶子至少需要一个新边端点，得到同样的下界。分解后本函数 $O(n+m)$ 时间与空间。依赖 `biconnected.hpp`。P2860 双风格完整程序本地验证；新增方案以逐边删除连通性独立验证，尚无本库在线 AC。

```

1 // Connected, nonempty graph after run(); new parallel edges are allowed.
2 inline vector<pair<int, int>> bridge_augmentation(const Biconnected &graph)
3 {
4     assert(graph.n > 0 && graph.cnt > 0);
5     int bridges = count(graph.bridge.begin(), graph.bridge.end(), 1);
6     assert(graph.cnt - bridges == 1);
7     if (graph.cnt == 1) return {};
8     auto tree = graph.bridge_forest();
9     vector<int> representative(graph.cnt + 1);
10    for (int u = 1; u <= graph.n; u++) representative[graph.bel[u]] = u;
11    vector<int> leaves;
12    auto dfs = [&](auto &&self, int u, int parent) -> void
13    {
14        if (tree[u].size() == 1) leaves.push_back(u);
15        for (auto [v, id] : tree[u])
16            if (v != parent) self(self, v, u);
17    };
18    dfs(dfs, 1, 0);
19    if (leaves.size() % 2) leaves.push_back(leaves[0]);
20    int half = leaves.size() / 2;
21    vector<pair<int, int>> added;
22    for (int i = 0; i < half; i++)
23        added.push_back({representative[leaves[i]], representative[leaves[i + half]]});
24    return added;
25 }

```

4.3.3 割点与桥

无向多重图，点号 $1..n$ ；`add/Insert(u,v)` 返回逻辑无向边的 0-based 编号，自环和重边均可输入。`run/Run` 后 `cut[u]` 为割点标记，`bridge[id]` 为桥标记，`components` 为原图连通块数。`delta[u]` 是独立删除顶点 u 及其关联边后增加的连通块数，剩余块数为 `components+delta[u]`；孤立点 `delta=-1`，普通非割点为 0，割点为正。根的增量为 DFS 子树数减 1；非根统计满足 `low[child]>=dfn[u]` 的子树数。 $n=1$ 的点删除后剩余 0 块。DFS 按父边号跳过，支持重边、非连通图和重复运行。复杂度 $O(n+m)$ ，递归 DFS；新增边后需重新 `run`。每个删点结果都针对原图，不表示逐次删除。

```

1 struct Lowlink
2 {
3     int n, timer = 0, components = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge, delta;
6     int edges = 0;
7
8     Lowlink(int n) : n(n), g(n + 1) {}
9
10    int add(int u, int v)
11    {
12        g[u].push_back({v, edges});
13        g[v].push_back({u, edges});
14        return edges++;
15    }

```

```

17 void dfs(int u, int pe)
18 {
19     dfn[u] = low[u] = ++timer;
20     int children = 0;
21     for (auto [v, id] : g[u])
22     {
23         if (id == pe) continue;
24         if (!dfn[v])
25         {
26             ++children;
27             dfs(v, id);
28             low[u] = min(low[u], low[v]);
29             if (low[v] > dfn[u]) bridge[id] = 1;
30             if (pe != -1 && low[v] >= dfn[u])
31             {
32                 cut[u] = 1;
33                 delta[u]++;
34             }
35         }
36         else
37             low[u] = min(low[u], dfn[v]);
38     }
39     if (pe == -1 && children > 1) cut[u] = 1;
40     if (pe == -1) delta[u] = children - 1;
41 }
42
43 void run()
44 {
45     timer = 0;
46     components = 0;
47     dfn.assign(n + 1, 0);
48     low = dfn;
49     cut = dfn;
50     delta = dfn;
51     bridge.assign(edges, 0);
52     for (int u = 1; u <= n; u++)
53         if (!dfn[u])
54         {
55             components++;
56             dfs(u, -1);
57         }
58 }
59 };

```

第五章 2-SAT

5.1 2-SAT 与任意可行方案

add(x,a,y,b) 添加 (x=a) 或 (y=b), 变量 1..n; solve() 返回可满足性, 仅成功时读取 ans。依赖同文件 SCC; $O(V+E)$, 不保证字典序最小。

```
1 struct TwoSAT
2 {
3     int n;
4     SCC g;
5     vector<int> ans;
6
7     TwoSAT(int n) : n(n), g(2 * n) {}
8
9     int id(int x, bool value) const { return 2 * x - 1 + value; }
10
11     void add(int x, bool a, int y, bool b)
12     {
13         g.add(id(x, !a), id(y, b));
14         g.add(id(y, !b), id(x, a));
15     }
16
17     bool solve()
18     {
19         g.run();
20         ans.assign(n + 1, 0);
21         for (int x = 1; x <= n; x++)
22         {
23             if (g.bel[id(x, 0)] == g.bel[id(x, 1)]) return false;
24             ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
25         }
26         return true;
27     }
28 };
```

5.2 递归染色与回滚求最小赋值

构造 LexTwoSAT(n)，变量号 $1..n$ 。add(x,a,y,b) 加入子句 $(x=a) \text{ OR } (y=b)$ ，force(x,v) 强制变量值；solve() 成功后 answer[$1..n$] 是按变量编号、0 优先于 1 的字典序最小赋值。失败时 answer 为空， $n=0$ 时可满足，answer[0] 始终仅为占位。每个未确定变量先尝试 0，递归传播所有蕴含；冲突时仅撤销本次新增标记，再试 1。history 是撤销记录，不替代递归调用栈。可重复求解，追加子句会清空旧答案；marked/history 仅为内部工作区，失败时不可当作方案。内部 id(x,v)= $2(x-1)+v$ ，从 0 开始，可用异或 1 取反；不要混用另一份 SCC 2-SAT 的 1-based 字面量编号。最坏 $O(n(n+m))$ 时间、 $O(n+m)$ 空间，递归栈 $O(n)$ ；100000 变量测试是特定蕴含链，不代表一般大规模输入的性能保证。HDU1814 完整程序本地对拍，在线待补。

```

1 struct LexTwoSAT
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> marked, history, answer;
6
7     LexTwoSAT(int n) : n(n), g(2 * n), marked(2 * n) { assert(n >= 0); }
8
9     int id(int x, bool value) const
10    {
11        assert(1 <= x && x <= n);
12        return 2 * (x - 1) + value;
13    }
14
15    // (x == a) OR (y == b). Internal literal IDs are 0-based.
16    void add(int x, bool a, int y, bool b)
17    {
18        int u = id(x, a), v = id(y, b);
19        g[u ^ 1].push_back(v);
20        g[v ^ 1].push_back(u);
21        answer.clear();
22    }
23
24    void force(int x, bool value) { add(x, value, x, value); }

```



```
26  bool paint(int u)
27  {
28      if (marked[u ^ 1]) return false;
29      if (marked[u]) return true;
30      marked[u] = 1;
31      history.push_back(u);
32      for (int v : g[u])
33          if (!paint(v)) return false;
34      return true;
35  }
36
37  bool solve()
38  {
39      fill(marked.begin(), marked.end(), 0);
40      history.clear();
41      answer.clear();
42      for (int x = 1; x <= n; x++)
43      {
44          int u = id(x, false);
45          if (marked[u] || marked[u ^ 1]) continue;
46          int checkpoint = history.size();
47          if (!paint(u))
48          {
49              while ((int)history.size() > checkpoint)
50              {
51                  marked[history.back()] = 0;
52                  history.pop_back();
53              }
54              if (!paint(u ^ 1)) return false;
55          }
56      }
57      answer.assign(n + 1, 0);
58      for (int x = 1; x <= n; x++) answer[x] = marked[id(x, true)];
59      return true;
60  }
61  };
```

第六章 欧拉图

6.1 有向图字典序最小欧拉路

构造 `DirectedEuler(n)`, $n \geq 1$, 点号 $1..n$; `add(u,v)` 返回从 0 开始的逻辑边编号, 支持自环与重边。`run()` 返回是否存在欧拉路; 参数 `start` 默认 0, `lexicographic` 默认 `true`。`start=0` 自动选择最小可行起点, 否则固定起点; 孤立点不影响有边部分的连通性。成功后 `vertices` 长度为 $m+1$, `edge_ids` 长度为 m , 第 i 条原边从 `vertices[i]` 指向 `vertices[i+1]`; 失败清空两数组。无边时返回单点路径, 自动起点为 1。可重复 `run`, `add` 会清空旧方案。默认求顶点序列字典序最小方案; 关闭排序后只保证任意方案, 邻接顺序可能保留上次排序。开启排序总时间 $O(n+m \log(m+1))$, 否则 $O(n+m)$, 空间 $O(n+m)$ 。DFS 递归深度可能达到 $m+1$, 不是 n 。度数检查后, 以消费边数判定是否遍历全部非孤立部分, 逆后序形成连续走法。小图全部走法对拍同时验证存在性、字典序及边编号, P7771 双风格完整程序本地验证, 在线待补。

```
1 struct DirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> in, out, next, vertices, edge_ids;
7
8     DirectedEuler(int n) : n(n), g(n + 1), in(n + 1), out(n + 1), next(n + 1)
9     {
10         assert(n > 0);
11     }
12
13     int add(int u, int v)
14     {
15         assert(1 <= u && u <= n && 1 <= v && v <= n);
16         int id = edges.size();
17         edges.push_back({u, v});
18         g[u].push_back({v, id});
19         out[u]++;
20         in[v]++;
21         vertices.clear();
22         edge_ids.clear();
23         return id;
24     }
25
26     void dfs(int u, int incoming)
27     {
28         while (next[u] < (int)g[u].size())
29         {
30             auto [v, id] = g[u][next[u]++];
31             dfs(v, id);
32         }
33         vertices.push_back(u);
34         if (incoming != -1) edge_ids.push_back(incoming);
35     }
```

```

37 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
38 bool run(int start = 0, bool lexicographic = true)
39 {
40     assert(0 <= start && start <= n);
41     vertices.clear();
42     edge_ids.clear();
43     int source = 0, sink = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         int delta = out[u] - in[u];
47         if (delta == 1 && !source)
48             source = u;
49         else if (delta == -1 && !sink)
50             sink = u;
51         else if (delta != 0)
52             return false;
53     }
54     if (bool(source) != bool(sink)) return false;
55     if (source)
56     {
57         if (start && start != source) return false;
58         start = source;
59     }
60     if (!start)
61     {
62         start = 1;
63         for (int u = 1; u <= n; u++)
64             if (out[u])
65             {
66                 start = u;
67                 break;
68             }
69     }
70     if (lexicographic)
71         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
72     fill(next.begin(), next.end(), 0);
73     dfs(start, -1);
74     if (edge_ids.size() != edges.size())
75     {
76         vertices.clear();
77         edge_ids.clear();
78         return false;
79     }
80     reverse(vertices.begin(), vertices.end());
81     reverse(edge_ids.begin(), edge_ids.end());
82     return true;
83 }
84 };

```

6.2 字典序最小单词链

`word_chain(words)` 返回可空的编号序列 (optional), 有解时为原输入的 0-based 单词编号顺序, 每个单词恰用一次; 相邻单词首尾字母相接, 单词序列字典序最小。无解返回 `nullopt`, 空输入返回有值的空序列。单词必须非空且仅含小写 a-z, 重复词视为不同编号。先对原编号按单词排序, 按此顺序加边, 再调用 `DirectedEuler.run(0,false)`; 不可让底层重新按终点排序。欧拉回路从最小出现首字母开始, 非回路起点由度数差确定。O(m log m) 次字符串比较及 O(L+m) 其他工作, L 为总字符数; 空间 O(m), 递归栈 O(m)。依赖 `directed_euler.hpp`。全排列对拍、重复词/前缀词、20 万相同词及 POJ2337 完整输出本地验证, 在线待补。

```

1 // Lowercase nonempty words. Returns original 0-based IDs; duplicates stay distinct.
2 inline optional<vector<int>> word_chain(const vector<string> &words)
3 {
4     int n = words.size();
5     vector<int> order(n);
6     iota(order.begin(), order.end(), 0);
7     for (const auto &word : words)
8     {
9         assert(!word.empty());
10        for (char c : word) assert('a' <= c && c <= 'z');
11    }
12    sort(order.begin(),
13         order.end(),
14         [&](int a, int b)
15         {
16             if (words[a] != words[b]) return words[a] < words[b];
17             return a < b;
18         });
19    DirectedEuler graph(26);
20    for (int id : order)
21        graph.add(words[id].front() - 'a' + 1, words[id].back() - 'a' + 1);
22    // Preserve word order in each adjacency list; do not sort by destination.
23    if (!graph.run(0, false)) return nullopt;
24    vector<int> result;
25    for (int id : graph.edge_ids) result.push_back(order[id]);
26    return result;
27 }
```

6.3 无向图欧拉路与边方向

构造 `UndirectedEuler(n)`, $n \geq 1$, 点号 $1..n$; `add(u,v)` 返回从 0 开始的逻辑边号, 自环和重边均保留。`run()` 返回是否存在欧拉路, 参数 `start` 默认 0 (自动), `lexicographic` 默认 `true`。自动选最小可行起点; 指定起点须满足奇度条件且位于有边部分。成功得到 $m+1$ 个 `vertices` 和 m 个 `edge_ids`, 失败清空两数组。第 i 条边方向是 `vertices[i]` 到 `vertices[i+1]`, 与 `edges[edge_ids[i]]` 比较可判断是否反向, 自环任选方向。`used` 按逻辑边号标记, 使两条半边共同只消费一次; 自环贡献度数 2。孤立点不影响有边部分; 无边图返回指定点或 1 的单点序列。默认求顶点序列字典序最小方案; 关闭排序只保证任意方案, 可能保留上次排序。可重复 `run`, 加边清空旧方案。排序开启时 $O(n+m \log(m+1))$, 否则 $O(n+m)$, 空间 $O(n+m)$, DFS 递归深度可能达到 $m+1$ 。全小图走法、原边号与方向独立验证, P2731 和 SGU101 完整程序本地验证; 在线待补。

```

1 struct UndirectedEuler
2 {
3     int n;
4     vector<vector<pair<int, int>>> g;
5     vector<pair<int, int>> edges;
6     vector<int> next, used, vertices, edge_ids;
7
8     UndirectedEuler(int n) : n(n), g(n + 1), next(n + 1) { assert(n > 0); }
9
10    int add(int u, int v)
11    {
12        assert(1 <= u && u <= n && 1 <= v && v <= n);
13        int id = edges.size();
14        edges.push_back({u, v});
15        g[u].push_back({v, id});
16        g[v].push_back({u, id});
17        vertices.clear();
18        edge_ids.clear();
19        return id;
20    }
21
22    void dfs(int u, int incoming)
23    {
24        while (next[u] < (int)g[u].size())
25        {
26            auto [v, id] = g[u][next[u]++];
27            if (used[id]) continue;
28            used[id] = 1;
29            dfs(v, id);
30        }
31        vertices.push_back(u);
32        if (incoming != -1) edge_ids.push_back(incoming);
33    }

```

```

35 // start=0 chooses the smallest feasible start. Isolated vertices are ignored.
36 bool run(int start = 0, bool lexicographic = true)
37 {
38     assert(0 <= start && start <= n);
39     vertices.clear();
40     edge_ids.clear();
41     vector<int> odd;
42     for (int u = 1; u <= n; u++)
43         if (g[u].size() % 2) odd.push_back(u);
44     if (!odd.empty() && odd.size() != 2) return false;
45     if (!odd.empty())
46     {
47         if (start && start != odd[0] && start != odd[1]) return false;
48         if (!start) start = odd[0];
49     }
50     if (!start)
51     {
52         start = 1;
53         for (int u = 1; u <= n; u++)
54             if (!g[u].empty())
55             {
56                 start = u;
57                 break;
58             }
59     }
60     if (lexicographic)
61         for (int u = 1; u <= n; u++) sort(g[u].begin(), g[u].end());
62     fill(next.begin(), next.end(), 0);
63     used.assign(edges.size(), 0);
64     dfs(start, -1);
65     if (edge_ids.size() != edges.size())
66     {
67         vertices.clear();
68         edge_ids.clear();
69         return false;
70     }
71     reverse(vertices.begin(), vertices.end());
72     reverse(edge_ids.begin(), edge_ids.end());
73     return true;
74 }
75 };

```

6.4 欧拉回路与指定端点定向

`mixed_euler_orientation(n,edges,start,finish)` 返回可空的定向边序列。每条输入为 $\{u,v,type\}$, $type=1$ 固定 u 到 v , $type=0$ 可反转; 点号 $1..n$, $n \geq 1$ 。省略两个端点或均传 0 时请求任意起点的欧拉回路; 两个正端点请求指定起终点的欧拉路径, 相等时为指定起点回路。无解返回 `nullopt`, 有解返回与原边顺序对应的端点对; 无边图可返回有值空序列。忽略无关孤立点, 但正端点须在有边部分。先任意定向可反转边, 扣除目标出入度差; 每条反转将两端差值分别改变 -2 和 $+2$, 奇数差无解, 否则用容量 1 的反转边流调整。检查底层无向连通性后, 满流即得到所需方向。最大流图 $n+2$ 点、至多 $m+n$ 条正向边, 总时间可按 Dinic 的 $O((n+2)^2(m+n))$ 上界计, 空间 $O(n+m)$ 。依赖 `flow.hpp`; 返回边交给 `DirectedEuler` 可生成连续走法, 指定端点时保持指定起点。定向不保证字典序最小。

```

1 // Edge = {u,v,type}: type 1 is fixed u->v; type 0 may be reversed.
2 // start=finish=0 requests a circuit; positive endpoints request a specified trail.
3 inline optional<vector<pair<int, int>>> mixed_euler_orientation(
4     int n, const vector<array<int, 3>> &edges, int start = 0, int finish = 0)
5 {
6     assert(n > 0 && 0 <= start && start <= n && 0 <= finish && finish <= n);
7     assert(bool(start) == bool(finish));
8     vector<int> parent(n + 1), id;
9     vector<long long> degree(n + 1), balance(n + 1);
10    iota(parent.begin(), parent.end(), 0);
11    auto find = [&](auto &&self, int u) -> int
12    {
13        if (parent[u] != u) parent[u] = self(self, parent[u]);
14        return parent[u];
15    };
16    Dinic flow(n + 2);
17    vector<pair<int, int>> direction;
18    for (auto [u, v, type] : edges)
19    {
20        assert(1 <= u && u <= n && 1 <= v && v <= n);
21        assert(type == 0 || type == 1);
22        parent[find(find, u)] = find(find, v);
23        degree[u]++;
24        degree[v]++;
25        balance[u]++;
26        balance[v]--;
27        id.push_back(type == 0 ? flow.add(u, v, 1) : -1);
28        direction.push_back({u, v});
29    }

```

```

30     if (!edges.empty())
31     {
32         int root = find(find, edges[0][0]);
33         for (int u = 1; u <= n; u++)
34             if (degree[u] && find(find, u) != root) return nullopt;
35         if (start && (!degree[start] || !degree[finish])) return nullopt;
36     }
37     if (start)
38     {
39         balance[start]--;
40         balance[finish]++;
41     }
42     int source = n + 1, sink = n + 2;
43     long long need = 0;
44     for (int u = 1; u <= n; u++)
45     {
46         if (balance[u] % 2) return nullopt;
47         if (balance[u] > 0)
48         {
49             flow.add(source, u, balance[u] / 2);
50             need += balance[u] / 2;
51         }
52         else if (balance[u] < 0)
53             flow.add(u, sink, -balance[u] / 2);
54     }
55     if (flow.flow(source, sink) != need) return nullopt;
56     for (int i = 0; i < (int)edges.size(); i++)
57         if (id[i] != -1 && flow.used(id[i]))
58             swap(direction[i].first, direction[i].second);
59     return direction;
60 }

```


6.5 任意起终点的混合欧拉路径

`mixed_euler_trail(n, edges)` 返回任意可行欧拉路径的定向边序列，格式同上一节。忽略边原方向统计度数，自环计 2；若奇度点为 0 个，请求回路；若恰有 2 个，尝试这两个端点的两种顺序；否则无解。最多两次定向求解。返回方向可交给 `DirectedEuler` 自动选择起点，恢复顶点与原边编号序列。枚举所有可反转边方向、全部固定端点及自由端点对拍；逐边检查固定边不变和连续走法，另有 10 万点长流量路径测试。POJ1637 双驱动本地验证，仅检查回路存在性；完整方向和开放路径依靠本地证据，在线待补。

```

1 // Any endpoints: zero odd degrees requests a circuit; two odd degrees tries both
2 // orders.
3 inline optional<vector<pair<int, int>>>
4 mixed_euler_trail(int n, const vector<array<int, 3>> &edges)
5 {
6     assert(n > 0);
7     vector<long long> degree(n + 1);
8     vector<int> odd;
9     for (auto [u, v, type] : edges)
10     {
11         assert(1 <= u && u <= n && 1 <= v && v <= n);
12         degree[u]++;
13         degree[v]++;
14     }
15     for (int u = 1; u <= n; u++)
16         if (degree[u] % 2) odd.push_back(u);
17     if (odd.empty()) return mixed_euler_orientation(n, edges);
18     if (odd.size() != 2) return nullopt;
19     auto answer = mixed_euler_orientation(n, edges, odd[0], odd[1]);
20     if (answer) return answer;
21     return mixed_euler_orientation(n, edges, odd[1], odd[0]);
22 }
```

第七章 网络流

7.1 最大流

7.1.1 最大流与方案

Dinic(n), add(u,v,c) 返回偶数原边编号; flow(s,t,limit) 返回本次新增流; used(id) 查询原边净流, cut(s) 返回残量可达点集。允许重边、自环, 容量非负, $s \neq t$; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$, 递归增广深度 $O(V)$ 。总流不得超过 long long 上限。get_edge(id) 返回 tuple(起点, 终点, 总容量, 当前流), change_edge(id,cap,f) 修改总容量与流, 要求 id 为 add 返回的偶数原边编号且 $0 \leq f \leq \text{cap}$; 正向残量变为 $\text{cap}-f$, 反向为 f 。仅保证此残量对一致, 不自动维持所有顶点的流量守恒; 继续 flow 前必须由调用方保证全局可行流及同源汇。增容且保留 f 合法, 缩容低于已有流时不能只截断该边。要从零重算须将所有原边一起重置为零流。两个新接口为 $O(1)$, 尚只有本地验证。

```
1 struct Dinic
2 {
3     using ll = long long;
4
5     struct Edge
6     {
7         int from, to;
8         ll cap, initial;
9     };
10
11     int n;
12     vector<Edge> e;
13     vector<vector<int>>> g;
14     vector<int> dep, cur;
15
16     Dinic(int n) : n(n), g(n + 1), dep(n + 1), cur(n + 1) {}
17
18     int add(int u, int v, ll c)
19     {
20         assert(c >= 0);
21         int id = (int)e.size();
22         e.push_back({u, v, c, c});
23         e.push_back({v, u, 0, 0});
24         g[u].push_back(id);
25         g[v].push_back(id + 1);
26         return id;
27     }
```

```

29  bool bfs(int s, int t)
30  {
31      fill(dep.begin(), dep.end(), -1);
32      queue<int> q;
33      dep[s] = 0;
34      q.push(s);
35      while (!q.empty())
36      {
37          int u = q.front();
38          q.pop();
39          for (int id : g[u])
40              if (e[id].cap && dep[e[id].to] < 0)
41              {
42                  dep[e[id].to] = dep[u] + 1;
43                  q.push(e[id].to);
44              }
45      }
46      return dep[t] >= 0;
47  }
48
49  ll dfs(int u, int t, ll f)
50  {
51      if (u == t) return f;
52      for (int &i = cur[u]; i < (int)g[u].size(); i++)
53      {
54          int id = g[u][i], v = e[id].to;
55          if (!e[id].cap || dep[v] != dep[u] + 1) continue;
56          ll d = dfs(v, t, min(f, e[id].cap));
57          if (d)
58          {
59              e[id].cap -= d;
60              e[id ^ 1].cap += d;
61              return d;
62          }
63      }
64      return 0;
65  }

```

```

67 // Returns additional flow; reuse only with the same source/sink.
68 ll flow(int s, int t, ll limit = LLONG_MAX)
69 {
70     assert(s != t && limit >= 0);
71     ll ans = 0;
72     while (ans < limit && bfs(s, t))
73     {
74         fill(cur.begin(), cur.end(), 0);
75         while (ans < limit)
76         {
77             ll d = dfs(s, t, limit - ans);
78             if (!d) break;
79             ans += d;
80         }
81     }
82     return ans;
83 }
84
85 ll used(int id) const { return e[id].initial - e[id].cap; }
86
87 tuple<int, int, ll, ll> get_edge(int id) const
88 {
89     assert(0 <= id && id < (int)e.size() && !(id & 1));
90     return {e[id].from, e[id].to, e[id].initial, used(id)};
91 }
92
93 // Changes one residual pair; caller must preserve global flow conservation.
94 void change_edge(int id, ll cap, ll f)
95 {
96     assert(0 <= id && id < (int)e.size() && !(id & 1));
97     assert(0 <= f && f <= cap);
98     e[id].initial = cap;
99     e[id].cap = cap - f;
100     e[id ^ 1].initial = 0;
101     e[id ^ 1].cap = f;
102 }
103
104 vector<int> cut(int s)
105 {
106     bfs(s, s);
107     vector<int> ans;
108     for (int u = 1; u <= n; u++)
109         if (dep[u] >= 0) ans.push_back(u);
110     return ans;
111 }
112 };

```

7.2 最小割

7.2.1 最大权闭合子图与选点方案

`maximum_closure(weight, dependencies)` 返回 (最大权值, 选中点列表), `weight` 为 0-based `vector<long long>`, 图中点号 `1..n`。依赖边 `(u,v)` 表示选 `u` 必须选 `v`, 允许环、自环与重边; 允许空选集, 因此答案不小于 0。依赖 `flow.hpp` 中的 `Dinic`。正权连源、负权连汇, 依赖边容量取正权总和加 1; 要求所有正权之和严格小于 `LLONG_MAX`。负权先提升到 `int128` 再取负, 并截断到依赖边容量, 因此 `LLONG_MIN` 也可处理。返回残量图源点可达的原顶点, 权值为正权总和减最小割; 超大负权边不会出现在最优割中, 截断不改变原问题最优值。每次调用重建网络。复杂度沿用 `Dinic`: 一般 $O(V^2E)$, 空间 $O(V+E)$, 增广保持递归。返回一种最优闭合集, 不承诺最大或最小选点个数。

```

1 inline pair<long long, vector<int>>
2 maximum_closure(const vector<long long> &weight,
3                 const vector<pair<int, int>> &dependencies)
4 {
5     using I = __int128_t;
6     int n = weight.size(), s = n + 1, t = n + 2;
7     I positive = 0;
8     for (long long w : weight)
9         if (w > 0) positive += w;
10    assert(positive < LLONG_MAX);
11    long long inf = (long long)positive + 1;
12    Dinic graph(n + 2);
13    for (int u = 1; u <= n; u++)
14    {
15        long long w = weight[u - 1];
16        if (w > 0)
17            graph.add(s, u, w);
18        else if (w < 0)
19            graph.add(u, t, (long long)min(-I(w), I(inf)));
20    }
21    for (auto [u, v] : dependencies)
22    {
23        assert(1 <= u && u <= n && 1 <= v && v <= n);
24        graph.add(u, v, inf);
25    }
26    long long answer = (long long)positive - graph.flow(s, t);
27    vector<int> selected;
28    for (int u : graph.cut(s))
29        if (u <= n) selected.push_back(u);
30    return {answer, selected};
31 }

```

7.2.2 Gomory–Hu 最小割树

`gomory_hu(n,edges)` 返回 $n-1$ 条 $(u,v, \text{容量})$ 树边； $n \geq 1$ ，原图点号 $0..n-1$ ，输入为无向边 tuple 列表，重边保留、自环忽略、容量非负。非自环边总容量须严格小于 `LLONG_MAX`。两点最小割是树路径最小边权；删去树边得到的两侧也是原图该边端点的最小割。允许非连通图和零权树边， $n=1$ 返回空树。运行 $n-1$ 次 Dinic，一般上界 $O(n^3m+n^2)$ ，额外空间 $O(n+m)$ ，DFS 增广保持递归。P4897 输入 n 表示 $n+1$ 个点，驱动传 $n+1$ 。双版本 P4897 已 AC，最长单点分别 252、185 ms。

```

1 inline vector<tuple<int, int, long long>>
2 gomory_hu(int n, const vector<tuple<int, int, long long>> &edges)
3 {
4     assert(n >= 1);
5     vector<int> parent(n);
6     vector<long long> value(n);
7     for (int s = 1; s < n; s++)
8     {
9         int t = parent[s];
10        Dinic flow(n);
11        for (auto [u, v, w] : edges)
12            if (u != v)
13            {
14                flow.add(u + 1, v + 1, w);
15                flow.add(v + 1, u + 1, w);
16            }
17        long long cut = flow.flow(s + 1, t + 1);
18        vector<bool> side(n);
19        for (int u : flow.cut(s + 1)) side[u - 1] = true;
20        for (int v = 1; v < n; v++)
21            if (v != s && parent[v] == t && side[v]) parent[v] = s;
22        if (side[parent[t]])
23        {
24            parent[s] = parent[t];
25            parent[t] = s;
26            value[s] = value[t];
27            value[t] = cut;
28        }
29        else
30            value[s] = cut;
31    }
32    vector<tuple<int, int, long long>> tree;
33    for (int v = 1; v < n; v++) tree.push_back({v, parent[v], value[v]});
34    return tree;
35 }

```

7.2.3 最小割树全点对查询

`cut_tree_values(n, tree)` 对合法的连通无环加权树计算全点对路径最小边权；点号 $0..n-1$ 、 $n \geq 1$ 、恰有 $n-1$ 条边，边权为非负 `long long`。输入可直接使用 `gomory_hu` 的返回值。结果 `ans[u][v]` 在 $u \neq v$ 时为两点最小割；对角线为 `LLONG_MAX` 哨兵，不能解释成同点最小割。 $n=1$ 返回单个哨兵。 $O(n^2)$ 时间与空间，递归 DFS 深度 $O(n)$ ，查询 $O(1)$ 。只需要标准库，不依赖流模板；不是任意图的查询函数。

```

1 inline vector<vector<long long>>
2 cut_tree_values(int n, const vector<tuple<int, int, long long>> &tree)
3 {
4     assert(n >= 1 && tree.size() == (size_t)n - 1);
5     vector<vector<pair<int, long long>>> g(n);
6     for (auto [u, v, w] : tree)
7     {
8         assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
9         g[u].push_back({v, w});
10        g[v].push_back({u, w});
11    }
12    vector<vector<long long>> answer(n, vector<long long>(n, LLONG_MAX));
13    for (int s = 0; s < n; s++)
14    {
15        auto dfs = [&](auto &&self, int u, int parent, long long value) -> void
16        {
17            answer[s][u] = value;
18            for (auto [v, w] : g[u])
19                if (v != parent) self(self, v, u, min(value, w));
20        };
21        dfs(dfs, s, -1, LLONG_MAX);
22    }
23    return answer;
24 }
```

7.3 费用流

7.3.1 势能最小费用最大流

$\text{add}(u, v, \text{cap}, \text{cost})$ 返回边编号, $\text{flow}(s, t, \text{limit})$ 返回新增流量及 128 位费用, $\text{used}(id)$ 返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化 $O(VE)$, 每次增广 $O(E \log V)$; 增广次数不能简单假设为 $O(V)$ 。另要求 $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$, 防止总费用超过 int128 。 $\text{slope}(s, t, \text{limit})$ 返回从 $(0, 0)$ 开始的本次新增流量/费用折点, 相邻同斜率段合并, 费用为 int128 。无增广时只返回起点; 每个整数流量的费用可在线性段内插值。重复调用仍用同源汇, 返回值不是历史累计值。 flow 不存储曲线; slope 额外空间为折点数。内部 run 不是用户接口。新曲线接口仅本地验证, P3381 驱动只输出终点, 不能据此验证全部中间折点。

```

1 struct MinCostFlow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5
6     struct Edge
7     {
8         int from, to;
9         ll cap, cost, initial;
10    };
11
12    int n;
13    vector<Edge> e;
14    vector<vector<int>>> g;
15
16    MinCostFlow(int n) : n(n), g(n + 1) {}
17
18    int add(int u, int v, ll cap, ll cost)
19    {
20        assert(cap >= 0 && cost != LLONG_MIN);
21        int id = (int)e.size();
22        e.push_back({u, v, cap, cost, cap});
23        e.push_back({v, u, 0, -cost, 0});
24        g[u].push_back(id);
25        g[v].push_back(id + 1);
26        return id;
27    }
28
29    ll used(int id) const { return e[id].initial - e[id].cap; }

```



```

31 // Negative costs allowed; initial residual graph must have no negative cycle.
32 pair<ll, i128> run(int s, int t, ll limit, vector<pair<ll, i128>> *curve)
33 {
34     assert(s != t && limit >= 0);
35     const i128 inf = i128(1) << 120;
36     vector<i128> h(n + 1, 0), d(n + 1);
37     vector<int> pre(n + 1);
38     // Super-source Bellman-Ford: feasible potentials for every vertex.
39     for (int it = 0; it < n; it++)
40     {
41         bool changed = false;
42         for (auto &a : e)
43             if (a.cap && h[a.to] > h[a.from] + a.cost)
44             {
45                 h[a.to] = h[a.from] + a.cost;
46                 changed = true;
47             }
48         if (!changed) break;
49         if (it == n - 1) throw invalid_argument("negative cycle");
50     }
51     ll f = 0;
52     i128 cost = 0;
53     optional<i128> last;
54     while (f < limit)
55     {
56         fill(d.begin(), d.end(), inf);
57         d[s] = 0;
58         priority_queue<pair<i128, int>,
59                        vector<pair<i128, int>>,
60                        greater<pair<i128, int>>>
61             q;
62         q.push({0, s});
63         while (!q.empty())
64         {
65             auto [du, u] = q.top();
66             q.pop();
67             if (du != d[u]) continue;
68             for (int id : g[u])
69             {
70                 auto &a = e[id];
71                 i128 nd = du + a.cost + h[u] - h[a.to];
72                 if (a.cap && nd < d[a.to])
73                 {
74                     d[a.to] = nd;
75                     pre[a.to] = id;
76                     q.push({nd, a.to});
77                 }
78             }
79         }

```

```

80         if (d[t] == inf) break;
81         for (int u = 1; u <= n; u++)
82             if (d[u] != inf) h[u] += d[u];
83         ll take = limit - f;
84         for (int u = t; u != s; u = e[pre[u]].from) take = min(take, e[pre[u]].cap);
85         for (int u = t; u != s; u = e[pre[u]].from)
86         {
87             int id = pre[u];
88             e[id].cap -= take;
89             e[id ^ 1].cap += take;
90             cost += i128(take) * e[id].cost;
91         }
92         f += take;
93         if (curve)
94         {
95             i128 unit = h[t] - h[s];
96             if (last && *last == unit) curve->pop_back();
97             curve->push_back({f, cost});
98             last = unit;
99         }
100     }
101     return {f, cost};
102 }
103
104 pair<ll, i128> flow(int s, int t, ll limit = LLONG_MAX)
105 {
106     return run(s, t, limit, nullptr);
107 }
108
109 vector<pair<ll, i128>> slope(int s, int t, ll limit = LLONG_MAX)
110 {
111     vector<pair<ll, i128>> curve{{0, 0}};
112     run(s, t, limit, &curve);
113     return curve;
114 }
115 };

```

7.4 上下界网络流

7.4.1 上下界可行循环流

$\text{add}(u, v, lo, hi)$, $0 \leq lo \leq hi$; $\text{solve}()$ 仅调用一次, 返回可行性, 仅成功后读取 $\text{used}(id)$ 。约束为每点流量守恒; 不是带源汇最大流。所有上下界及其总和须在 long long 范围内。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll = long long;
4     int n;
5     Dinic g;
6     vector<ll> balance, lower;
7     vector<int> ids;
8     bool solved = false, feasible = false;
9
10    BoundedCirculation(int n) : n(n), g(n + 2), balance(n + 1) {}
11
12    int add(int u, int v, ll lo, ll hi)
13    {
14        assert(!solved && 0 <= lo && lo <= hi);
15        int id = ids.size();
16        ids.push_back(g.add(u, v, hi - lo));
17        lower.push_back(lo);
18        balance[u] -= lo;
19        balance[v] += lo;
20        return id;
21    }
22
23    bool solve()
24    {
25        assert(!solved);
26        solved = true;
27        ll need = 0;
28        for (int u = 1; u <= n; u++)
29            if (balance[u] > 0)
30            {
31                g.add(n + 1, u, balance[u]);
32                need += balance[u];
33            }
34            else if (balance[u] < 0)
35                g.add(u, n + 2, -balance[u]);
36        return feasible = (g.flow(n + 1, n + 2) == need);
37    }
38
39    ll used(int id) const
40    {
41        assert(feasible);
42        return lower[id] + g.used(ids[id]);
43    }
44 };

```

7.5 Stoer–Wagner 算法

7.5.1 无向全局最小割

$\text{solve}(w)$, $n \geq 2$; w 为 $n \times n$ 对称非负权矩阵, 重边先累加, 主对角置零。返回 {割权, 一侧顶点列表}, 编号 $0..n-1$; 两侧非空, 不要求原图连通, 不连通时割权为 0。并列最优割返回其中一种, 不承诺点数或字典序。矩阵按值传入, 原图不变。 $O(n^3)$ 时间、 $O(n^2)$ 空间; 无向边总权 (每条只计一次) 严格小于 2^{120} , 内部 int_{128} 。负权不适用。已逐条核验返回点集在原图上的跨割费用, 暂无专门 OJ AC。

```

1 struct StoerWagner
2 {
3     using I = __int128_t;
4
5     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
6     static pair<I, vector<int>> solve(vector<vector<I>> w)
7     {
8         int n = w.size();
9         assert(n >= 2);
10        vector<int> active(n);
11        iota(active.begin(), active.end(), 0);
12        vector<vector<int>> group(n);
13        for (int i = 0; i < n; i++) group[i] = {i};
14        I best = I(1) << 120;
15        vector<int> side;
16        while (active.size() > 1)
17        {
18            vector<I> dis(n);
19            vector<bool> used(n);
20            int prev = -1;
21            for (int step = 0; step < (int)active.size(); step++)
22            {
23                int sel = -1;
24                for (int v : active)
25                    if (!used[v] && (sel == -1 || dis[v] > dis[sel])) sel = v;
            }
        }
    }
}

```

本轮割、合并与继续扩张（接上页同一函数）：

```
26         if (step + 1 == (int)active.size())
27         {
28             if (dis[sel] < best)
29             {
30                 best = dis[sel];
31                 side = group[sel];
32             }
33             for (int v : active)
34                 if (v != prev && v != sel)
35                     w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
36             group[prev].insert(
37                 group[prev].end(), group[sel].begin(), group[sel].end());
38             active.erase(find(active.begin(), active.end(), sel));
39             break;
40         }
41         used[sel] = true;
42         prev = sel;
43         for (int v : active)
44             if (!used[v]) dis[v] += w[sel][v];
45     }
46 }
47 return {best, side};
48 }
49 };
```

第八章 图的匹配

8.1 二分图最大匹配

8.1.1 Hopcroft–Karp 最大匹配与最小点覆盖

左右两侧分别编号 $1..n$ 、 $1..m$ ，可有空侧及重边； $\text{add}(u,v)$ 加边， $\text{solve}()$ 返回最大匹配总数， $l[u]$ 与 $r[v]$ 为匹配端点，0 表示未匹配。保留已有合法匹配，可加边后再次 solve ；不支持删边或手动更改匹配数组。BFS 用 $\text{dep}[0]$ 记录最短终点层，递归 DFS 只沿最短层增广，并使用当前弧。 $V=n+m$ ，单次 solve 最坏 $O((V+E)\sqrt{V})$ ，空间 $O(V+E)$ 。 $\text{cover}()$ 在 solve 后、下一次改图前调用，返回 (左侧点列表, 右侧点列表)，为最小点覆盖；其补集是最大独立集。覆盖与匹配方案均可直接输出。

```
1 struct BipartiteMatching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep, cur;
6
7     BipartiteMatching(int n, int m)
8         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1), cur(n + 1)
9     {
10    }
11
12    void add(int u, int v)
13    {
14        assert(1 <= u && u <= n && 1 <= v && v <= m);
15        g[u].push_back(v);
16    }
17
18    bool bfs()
19    {
20        queue<int> q;
21        fill(dep.begin(), dep.end(), n + 1);
22        fill(cur.begin(), cur.end(), 0);
23        for (int u = 1; u <= n; u++)
24        {
25            if (!l[u])
26            {
27                dep[u] = 0;
28                q.push(u);
29            }
30        }
31        while (!q.empty())
32        {
33            int u = q.front();
34            q.pop();
35            if (dep[u] >= dep[0]) continue;
36            for (int v : g[u])
37            {
38                int next = r[v];
39                if (dep[next] != n + 1) continue;
40                dep[next] = dep[u] + 1;
```

```

41         if (next) q.push(next);
42     }
43 }
44 return dep[0] != n + 1;
45 }
46
47 bool dfs(int u)
48 {
49     if (!u) return true;
50     for (int &i = cur[u]; i < (int)g[u].size(); i++)
51     {
52         int v = g[u][i];
53         if (dep[r[v]] == dep[u] + 1 && dfs(r[v]))
54         {
55             l[u] = v;
56             r[v] = u;
57             return true;
58         }
59     }
60     dep[u] = n + 1;
61     return false;
62 }
63
64 // Retains an existing matching; edges may be added between calls.
65 int solve()
66 {
67     int answer = 0;
68     for (int u = 1; u <= n; u++) answer += (l[u] != 0);
69     while (bfs())
70     {
71         for (int u = 1; u <= n; u++)
72         {
73             if (!l[u] && dfs(u)) answer++;
74         }
75     }
76     return answer;
77 }
78
79 // Call after solve(), before adding more edges. Returns left/right vertex IDs.
80 pair<vector<int>, vector<int>> cover() const
81 {
82     vector<int> left, right;
83     for (int u = 1; u <= n; u++)
84     {
85         if (dep[u] == n + 1) left.push_back(u);
86     }
87     for (int v = 1; v <= m; v++)
88     {
89         if (r[v] && dep[r[v]] != n + 1) right.push_back(v);
90     }
91     return {left, right};
92 }
93 };

```

8.2 二分图最大权匹配

8.2.1 矩形二分图最小权匹配

`solve(cost)` 输入 $n \times m$ 完整权值矩阵, $0 \leq n \leq m$; 每行匹配不同列, 返回最小费用及列号 (0-based)。允许负费用, 使用 `int128` 势能; $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代, 当前接口仅表示完整二分图。

```

1 struct Hungarian
2 {
3     using I = __int128_t;
4
5     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
6     static pair<I, vector<int>> solve(const vector<vector<long long>> &cost)
7     {
8         int n = cost.size();
9         if (!n) return {0, {}};
10        int m = cost[0].size();
11        assert(n <= m);
12        vector<I> u(n + 1), v(m + 1);
13        vector<int> p(m + 1), way(m + 1);
14        const I inf = I(1) << 120;
15        for (int i = 1; i <= n; i++)
16        {
17            assert((int)cost[i - 1].size() == m);
18            p[0] = i;
19            int j0 = 0;
20            vector<I> dis(m + 1, inf);
21            vector<bool> used(m + 1);
22            do
23            {
24                used[j0] = true;
25                int i0 = p[j0], j1 = 0;
26                I delta = inf;
27                for (int j = 1; j <= m; j++)
28                    if (!used[j])
29                    {
30                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
31                        if (cur < dis[j])
32                        {
33                            dis[j] = cur;
34                            way[j] = j0;
35                        }
36                        if (dis[j] < delta)
37                        {
38                            delta = dis[j];
39                            j1 = j;
40                        }
41                    }
42                for (int j = 0; j <= m; j++)
43                    if (used[j])
44                    {
45                        u[p[j]] += delta;
46                        v[j] -= delta;
47                    }
48                else
49                    dis[j] -= delta;

```



```

50         j0 = j1;
51     } while (p[j0]);
52     do
53     {
54         int j1 = way[j0];
55         p[j0] = p[j1];
56         j0 = j1;
57     } while (j0);
58 }
59 vector<int> ans(n);
60 for (int j = 1; j <= m; j++)
61     if (p[j]) ans[p[j] - 1] = j - 1;
62 return {-v[0], ans};
63 }
64 };

```

8.2.2 二分图最大权匹配（缺边与可选匹配）

WeightedMatching(n,m)，两侧编号 1..n、1..m。add(x,y,w) 加边，重边保留最大权；边权支持完整 long long 范围，内部势能与答案为 int128。solve() 要求全部左点匹配不同右点，无解返回 nullopt；n=m 时即最大权完美匹配。solve(true) 允许点不匹配，只最大化总权，不优先最大化边数。两种模式均从空匹配重算；l[x]、r[y] 为方案，0 表示未匹配，无解时全部清零。缺边用独立于合法边权的 -2^{120} 哨兵，仅用 add 修改；不把缺边当零权边。允许空侧。全匹配 $O(n^2m)$ ，可选模式增加 n 个虚拟零权右点，为 $O(n^2(m+n))$ ；空间 $O(nm+n+m)$ 。按最短松弛量扩展交错树：每次至少访问一个新右点，最多 n 次后增广；无有限松弛量即无解。

```

1 struct WeightedMatching
2 {
3     using I = __int128_t;
4     static constexpr I inf = I(1) << 120;
5     int n, m;
6     vector<vector<I>> w;
7     vector<int> l, r;
8
9     WeightedMatching(int n, int m)
10         : n(n), m(m), w(n + 1, vector<I>(m + 1, -inf)), l(n + 1), r(m + 1)
11     {
12     }
13
14     void add(int x, int y, long long weight)
15     {
16         assert(1 <= x && x <= n && 1 <= y && y <= m);
17         w[x][y] = max(w[x][y], I(weight));
18     }
19
20     optional<I> solve(bool allow_unmatched = false)
21     {
22         fill(l.begin(), l.end(), 0);
23         fill(r.begin(), r.end(), 0);
24         if (!allow_unmatched && n > m) return nullopt;
25         int cols = m + (allow_unmatched ? n : 0);
26         vector<I> u(n + 1), v(cols + 1);
27         vector<int> p(cols + 1), way(cols + 1);
28         for (int i = 1; i <= n; i++)
29         {
30             p[0] = i;

```

```

31     int j0 = 0;
32     vector<I> dis(cols + 1, inf);
33     vector<bool> used(cols + 1);
34     do
35     {
36         used[j0] = true;
37         int x = p[j0], j1 = 0;
38         I delta = inf;
39         for (int j = 1; j <= cols; j++)
40         {
41             if (used[j]) continue;
42             I weight = j <= m ? w[x][j] : 0;
43             if (weight != -inf)
44             {
45                 I cur = -weight - u[x] - v[j];
46                 if (cur < dis[j])
47                 {
48                     dis[j] = cur;
49                     way[j] = j0;
50                 }
51             }
52             if (dis[j] < delta)
53             {
54                 delta = dis[j];
55                 j1 = j;
56             }
57         }
58         if (delta == inf) return nullopt;
59         for (int j = 0; j <= cols; j++)
60         {
61             if (used[j])
62             {
63                 u[p[j]] += delta;
64                 v[j] -= delta;
65             }
66             else if (dis[j] != inf)
67                 dis[j] -= delta;
68         }
69         j0 = j1;
70     } while (p[j0]);
71     do
72     {
73         int j1 = way[j0];
74         p[j0] = p[j1];
75         j0 = j1;
76     } while (j0);
77 }
78 I answer = 0;

```

```

79     for (int j = 1; j <= m; j++)
80     {
81         if (!p[j]) continue;
82         l[p[j]] = j;
83         r[j] = p[j];
84         answer += w[p[j]][j];
85     }
86     return answer;
87 }
88 };

```

8.3 一般图最大匹配

8.3.1 一般图最大基数匹配：带花树

Blossom(n), 点号 $0..n-1$; add(u,v) 加无向边, 忽略自环, 允许重边。solve() 从空匹配重新计算, 返回匹配边数; mate[u] 为配对点, 未匹配为 -1 。结果支持直接输出方案。缩花维护交错树, 单次增广 $O(n^2+m)$, 总计 $O(n^3+nm)$, 简单图为 $O(n^3)$; 空间 $O(n+m)$ 。重复调用 solve 或加边后重算均可。不处理边权, 不能代替最大权匹配。

```

1 struct Blossom
2 {
3     int n;
4     vector<vector<int>> g;
5     vector<int> mate, pre, base, q;
6     vector<bool> outer, flower, seen;
7
8     Blossom(int n)
9         : n(n), g(n), mate(n, -1), pre(n), base(n), outer(n), flower(n), seen(n)
10    {
11    }
12
13    void add(int u, int v)
14    {
15        assert(0 <= u && u < n && 0 <= v && v < n);
16        if (u == v) return;
17        g[u].push_back(v);
18        g[v].push_back(u);
19    }
20
21    int lca(int u, int v)
22    {
23        fill(seen.begin(), seen.end(), false);
24        while (true)
25        {
26            u = base[u];
27            seen[u] = true;
28            if (mate[u] == -1) break;
29            u = pre[mate[u]];
30        }
31        while (!seen[base[v]]) v = pre[mate[base[v]]];
32        return base[v];
33    }
34
35    void mark(int u, int b, int v)
36    {

```

```

37     while (base[u] != b)
38     {
39         flower[base[u]] = true;
40         flower[base[mate[u]]] = true;
41         pre[u] = v;
42         v = mate[u];
43         u = pre[v];
44     }
45 }
46
47 bool augment(int root)
48 {
49     fill(outer.begin(), outer.end(), false);
50     fill(pre.begin(), pre.end(), -1);
51     iota(base.begin(), base.end(), 0);
52     q.assign(1, root);
53     outer[root] = true;
54     for (int head = 0; head < (int)q.size(); head++)
55     {
56         int u = q[head];
57         for (int v : g[u])
58         {
59             if (base[u] == base[v] || mate[u] == v) continue;
60             if (v == root || (mate[v] != -1 && pre[mate[v]] != -1))
61             {
62                 int b = lca(u, v);
63                 fill(flower.begin(), flower.end(), false);
64                 mark(u, b, v);
65                 mark(v, b, u);
66                 for (int x = 0; x < n; x++)
67                 {
68                     if (!flower[base[x]]) continue;
69                     base[x] = b;
70                     if (!outer[x])
71                     {
72                         outer[x] = true;
73                         q.push_back(x);
74                     }
75                 }
76             }
77             else if (pre[v] == -1)
78             {
79                 pre[v] = u;
80                 if (mate[v] == -1)
81                 {
82                     while (v != -1)
83                     {
84                         int p = pre[v];
85                         int next = mate[p];
86                         mate[v] = p;
87                         mate[p] = v;
88                         v = next;
89                     }
90                     return true;
91                 }
92                 int x = mate[v];

```

```
93         outer[x] = true;
94         q.push_back(x);
95     }
96 }
97 }
98 return false;
99 }
100
101 // Rebuild the maximum matching; mate[u] is -1 for an unmatched vertex.
102 int solve()
103 {
104     fill(mate.begin(), mate.end(), -1);
105     int answer = 0;
106     for (int u = 0; u < n; u++)
107     {
108         if (mate[u] == -1 && augment(u)) answer++;
109     }
110     return answer;
111 }
112 };
```

第九章 矩阵树定理

9.1 带权矩阵树：无向树与内外向树形图

声明 `MatrixTree<mod>`，调用 `count(n, edges, root, kind)` 返回所有合法树的边权乘积之和，`mod` 必须是素数。`Edge=(u,v,Z)`，点号 `0..n-1`， $n \geq 1$ ；`Kind::undirected` 为无向生成树，`toward_root` 为每点沿边能到根，`away_from_root` 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 `root` 决定答案。依赖 `det_prime` 的素数模行列式， $O(n^3+m)$ 时间、 $O(n^2)$ 额外空间。

```
1 template <int mod> struct MatrixTree
2 {
3     using Z = ModInt<mod>;
4     using Edge = tuple<int, int, Z>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
13    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
14    {
15        assert(n >= 1 && root >= 0 && root < n);
16        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
17        auto add = [&](int u, int v, Z w)
18        {
19            if (u == root) return;
20            int i = u - (u > root), j = v - (v > root);
21            lap[i][i] = lap[i][i] + w;
22            if (v != root) lap[i][j] = lap[i][j] - w;
23        };
24        for (auto [u, v, w] : edges)
25        {
26            assert(u >= 0 && u < n && v >= 0 && v < n);
27            if (u == v) continue;
28            if (kind == Kind::away_from_root) swap(u, v);
29            add(u, v, w);
30            if (kind == Kind::undirected) add(v, u, w);
31        }
32        return det_prime<mod>(move(lap));
33    }
34};
```

9.2 任意正模数带权矩阵树

`count(n, edges, root, kind, mod)` 返回树的边权乘积之和对 `mod` 的最小非负余数。`mod` 为任意正 signed 64 位数，边权支持完整 signed 64 位。`Kind` 的 `undirected`、`toward_root`、`away_from_root` 分别表示无向、内向和外向树；点号 `0..n-1`。自环忽略，重边按不同边计数，单点返回 `1%mod`，`mod=1` 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 `determinant_mod`； $O(n^3 \log \text{mod} + m)$ 时间、 $O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 `MatrixTree<mod>`。

```

1 struct MatrixTreeMod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if (u == root) return;
19            int i = u - (u > root), j = v - (v > root);
20            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
21            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
22        };
23        for (auto [u, v, w] : edges)
24        {
25            assert(u >= 0 && u < n && v >= 0 && v < n);
26            if (u == v) continue;
27            w %= mod;
28            if (w < 0) w += mod;
29            if (kind == Kind::away_from_root) swap(u, v);
30            add(u, v, w);
31            if (kind == Kind::undirected) add(v, u, w);
32        }
33        return determinant_mod(move(lap), mod);
34    }
35 };

```

第十章 配套知识与验证范围

10.1 非负权树的点集直径摘要

代码见第 1 页。一个非空点集只需保存任意一对直径端点 a, b ，即可回答任意树上点 x 到该点集的最远距离： $\max(d(x, a), d(x, b))$ 。边权非负，包括零权；该结论不能推广到一般图或负权树。

证明可将点 u 投影到路径 $a-b$ ，记投影距 a 为 t ，离路径的距离为 h ，直径长为 D 。由于 $d(a, u), d(b, u) \leq D$ ，有 $h \leq \min(t, D - t)$ 。再将查询点投影到同一路径，投影距 a 为 s ，离路径的距离为 g 。经两个投影点连接得到 $d(x, u) \leq g + h + |s - t| \leq g + \max(s, D - s)$ ，右侧恰为 $\max(d(x, a), d(x, b))$ 。因此插入新点只需比较到两个旧端点的距离；合并点集时，只插入另一个摘要的两个端点即可。并列直径可任选，不应假设端点编号固定或合并后的端点与合并顺序无关。

Euler LCA 现在同时保存根距离，但其原有 `distance` 接口仍返回边数。带权距离请显式调用 `weighted_distance / Weighted_Distance`；采用 $(D_u - D_z) + (D_v - D_z)$ 避免不必要的两倍根距离中间值。所有路径权和须能用 `long long` 表示。独立验证穷举小点集及合并，逐点检查最远距离与返回顶点；带权路径用独立树遍历参照，并覆盖全部根、零边权与 `LLONG_MAX` 单边。原有 Euler LCA 全小树回归，以及 50 万点、边权 10^9 的递归链也通过普通与 ASan/UBSan 检查。

杭州 2023 F 中，点权互异。令 g 为全树点权的 `mex`，只需按点权 $0, 1, \dots, g - 1$ 建立前缀摘要。某个前缀全部位于以 x 为中心、半径 k 的范围内，当且仅当该摘要的最远距离不超过 k 。二分最长满足条件的前缀长度即为答案； $g = 0$ 时直接为零。题面树边带权，不能用边数距离。完整驱动在 `verify/qoj/8235.*.cpp`，本地使用实际权距与 `mex` 集合参照， $n = q = 500000$ 的链和星形树保持递归 DFS。两套驱动各 50 个正式测试和 1 个额外测试全部在线 AC：动态版 666 ms、198076 KB，传统版 588 ms、198196 KB。OJ 验证题面正权边、前缀插入和最远距离判定；零权边、完整 64 位距离与任意摘要合并仍由独立本地验证支持。

10.2 函数图的人环链与到达查询

代码见第 27 页。每个点有且仅有一个后继，故每个弱连通分量由一个有向环和若干指向环的树组成。先不断移除入度为零的点，剩下的点恰在环上；沿后继枚举各环。反向处理移除序列时，后继的环编号、入环点和入环距离均已确定，可直接传给当前点。这里使用的是入度剥离队列，不是用显式栈模拟 DFS。

跳转 k 步时，若 k 小于入环距离，用普通倍增；否则先到入环点，减去入环步数，再对环长取模。因此即使 k 达到无符号 64 位上界，也只需 $O(\log n)$ 层倍增表。环上位置相加使用无符号宽整数，避免两个大下标的 `int` 中间值溢出。

判断从 u 到 v 的最少步数时：不同环编号必不可达。若 v 在入环树上，先检查 u 的深度至少为 v 的深度，再跳过深度差，看是否恰好到 v ，从而排除同分量的不同分支。若 v 在环上，答案为 u 的入环步数加上沿后继方向的环上距离。不可达返回 -1 ，从自己到自己为零步。字段 `position` 只对环点有意义，不要将树点的 -1 当作环坐标。

独立参照逐步行走直至首次重复，同时记录每个点的第一次出现位置。穷举不超过 5 个点的全部后继映射，并检查跳转、最少步数、入环点与环长；另验证同对象重建、空图、50 万点长链连环、50 万个自环和完整无符号 64 位步数。普通与 ASan/UBSan 均通过。完整驱动对应 [CSES 1750](#)、[CSES 1160](#) 和 [P2921](#)，均准备两套码风。前两题检查跳转与可达步数，后一题仅使用“入环步数 + 环长”，不能替代查询接口的验证。线上评测尚待完成。

10.3 杭州 2023 H：随机事件顺序与函数图

[Sugar Sweet II](#) 的完整驱动为 `verify/qoj/8237/*.cpp`，复用函数图分解与逆阶乘表。事件 u 发生前，自己的数量仍为 a_u ；目标 $v = b_u$ 只可能为 a_v 或 $a_v + w_v$ 。因此 $a_u < a_v$ 时必定发生， $a_u \geq a_v + w_v$ 时必定不发生。其余情况下，事件 u 成功当且仅当事件 v 更早执行且成功。

将两类确定事件的后继改为自己，另保存哪些是确定成功的终点。未修改的依赖环没有成功起点：最早执行的环上事件必失败，后续也无法形成成功链。自环同理，自己不可能严格少于自己。沿函数图到达确定失败终点或纯依赖环的节点，成功概率都是零。若到达确定成功终点，且入环距离为 d ，沿途 $d + 1$ 个事件必须按唯一的顺序出现。均匀随机全排列对这 $d + 1$ 个事件诱导均匀相对顺序，因此成功概率为 $1/(d + 1)!$ 。这些顺序约束有关联，不能将每条依赖当作独立的二分之一概率。期望为 $a_u + w_u/(d + 1)!$ ，在模 $10^9 + 7$ 下计算。

题面所有增量为正， $\sum n \leq 500000$ ；阶乘表跨测试组只扩充不缩小，避免大量单点测试时反复求逆。当前驱动复用带倍增表的通用函数图，时间、空间为 $O(n \log n)$ ；概率传播本身只读取 `entry` 和 `depth`，不调用跳转与最少步数接口。

验证穷举小输入及事件全排列，逐步修改真实糖果数量，最后汇总精确总和并除以排列数。另外检查 50 万点的依赖链、纯环，以及 50 万组单点测试，普通与 ASan/UBSan 均通过。两套驱动各 31 个正式测试与 2 个额外测试全过：[动态版](#) 162 ms、84608 KB，[传统版](#) 158 ms、84612 KB。本应用不能验证函数图的 `advance` 或 `steps` 接口。

10.4 杭州 2023 G：开放时间的单位边最短路

驱动 `verify/qoj/8236*.cpp` 直接枚举网格邻居，不建立四倍规模的边表。初始蛇身从头到尾编号 $0, \dots, k-1$ ，第 $i \geq 1$ 节所在点的最早进入时间为 $k-i$ ，头与其他空格为零。旧蛇尾允许在同一步进入，所以尾格的限制必须是 1。长度为 4 的满 2×2 网格样例距离为 0, 3, 2, 1，平方和为 14。

每步只能删除或移动一个尾节，故这些时间限制必要。最短路可以取不重复顶点的路径；行走前把途中等待的总时间全部用于缩短蛇身，随后沿路径连续移动。进入初始蛇身第 i 节时，已删除的尾节数量至少为 $k-i$ ；新形成的蛇身不会与无重复顶点的路径相撞。因此这些限制也充分。等待超过 $k-1$ 没有必要，因为此时全部初始蛇身均已释放。

转移为 $d_v = \max(d_u + 1, \text{open}_v)$ 。按距离递增处理顶点时，第一个发现 v 的前驱就给出最小候选值，故不需要后续松弛。候选等于 $d_u + 1$ 的顶点进入普通 FIFO；其余进入对应开放时间的桶。每次取两者中距离较小的一项。桶指针只能扫描到下一个 FIFO 距离，否则可能越过之后才被发现较早开放时间。令 $R = \max \text{open}$ ，复杂度 $O(V + E + R)$ ，空间 $O(V + R)$ 。此性质依赖所有边权为 1；一般非负权仍需优先队列。

本地以独立堆 Dijkstra 对拍，并穷举 2×3 网格障碍和全部 744 种初始蛇形，对真实蛇身序列逐步执行移动与缩尾 BFS，核对每个点的最早到达时间。另验证正式样例、满长链和 3000×3000 空网格及隔断网格；普通与 ASan/UBSan 均通过。答案用 unsigned long long 累加距离平方，自然实现模 2^{64} 。两套驱动在 QOJ8236 各通过 58 个正式与 3 个额外测试：动态版 324 ms、123080 KB，传统版 294 ms、122768 KB。线上证据对应单位网格、源点时间为零及初始蛇身开放时间；一般有向图与非零源点时间由本地对拍覆盖。

10.5 SCC 模板题与速度榜口径

QOJ906 来自 Library Checker，要求输出强连通分量的拓扑序。本库 Tarjan 编号按逆拓扑序产生，驱动将输入点号加一，再按分量编号从大到小输出，输出点号减一。CCF_NOI 的记录 2940638 通过全部 11 个测试，203 ms、58636 KB。验证覆盖 add、run、bel 和完整分量输出，不调用 dag。本地独立可达性参考核对分量等价关系及每条跨分量边方向，并测试 50 万点链、环和多分量。普通与 ASan/UBSan 均通过；本地测试线程使用 1 GiB 栈容纳 sanitizer 扩大的递归栈帧，提交代码保持普通 main 和递归 DFS。

2026-09-12 核对速度榜时，页面第 66 行处于 203 ms 的第 65–67 行并列区间。但该表合并同一用户的提交，不能将行号与满分提交数 578 配对。以 CCF_NOI 逐页读取页面链接的满分列表，共取得 143 条，其中 93 条严格更快、3 条同为 203 ms。可见范围内按耗时并列排名为 94 out of 143，位置区间为 94–96。统计页显示所有提交 1053、满分提交 578；与可见列表的计数差异原因尚未核实，因此全站排名仍为待查。记录清单与证据见 docs/ranking-audits.json，不得将该局部排名改写为全站排名。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```
1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >>= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };
```

附录 B 线性代数

B.1 行列式

B.1.1 素数模行列式

`det_prime<p>(a)` 输入 $n \times n$ 方阵, 返回行列式模 p ; p 必须是素数。允许奇异矩阵和零维矩阵, `det(empty)=1`。按行交换改变符号, 主元求逆消元。 $O(n^3)$ 时间、传值副本 $O(n^2)$ 空间, 可传 `move(a)`。合数模数改用 `determinant_mod`。

```
1 template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
2 {
3     using Z = ModInt<mod>;
4     int n = (int)a.size();
5     Z ans = 1;
6     for (int i = 0; i < n; i++)
7     {
8         assert((int)a[i].size() == n);
9         int p = i;
10        while (p < n && !a[p][i].v) ++p;
11        if (p == n) return 0;
12        if (p != i)
13        {
14            swap(a[p], a[i]);
15            ans = Z(0) - ans;
16        }
17        ans = ans * a[i][i];
18        Z inv = a[i][i].inv();
19        for (int j = i + 1; j < n; j++)
20        {
21            Z f = a[j][i] * inv;
22            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
23        }
24    }
25    return ans;
26 }
```

B.1.2 任意正模数行列式：欧几里得消元

`determinant_mod(a,mod)` 返回 $\det(a)$ 的最小非负余数。 a 为 `long long` 方阵，元素支持完整 signed 64 位； $1 \leq \text{mod} \leq \text{LLONG_MAX}$ ，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 $1 \% \text{mod}$ ， $\text{mod}=1$ 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 `int128`。 $O(n^3 \log \text{mod})$ 时间、 $O(n^2)$ 空间；素数模通常优先用已有高斯行列式。

```

1 inline long long determinant_mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for (auto &row : a)
6     {
7         assert((int)row.size() == n);
8         for (auto &x : row)
9         {
10             x %= mod;
11             if (x < 0) x += mod;
12         }
13     }
14     long long answer = 1 % mod;
15     for (int i = 0; i < n; i++)
16     {
17         for (int j = i + 1; j < n; j++)
18             while (a[j][i])
19             {
20                 long long quotient = a[i][i] / a[j][i];
21                 for (int k = i; k < n; k++)
22                 {
23                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
24                     value %= mod;
25                     if (value < 0) value += mod;
26                     a[i][k] = (long long)value;
27                 }
28                 swap(a[i], a[j]);
29                 answer = answer ? mod - answer : 0;
30             }
31         if (a[i][i] == 0) return 0;
32         answer = (__int128)answer * a[i][i] % mod;
33     }
34     return answer;
35 }

```

附录 C 树状数组

C.1 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。T 须能从 0 构造并支持 $+=$ 与减法；可直接使用 `ModInt` 维护模数下的区间和。 kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```


算法索引

Arborescence, 24

Biconnected, 32

BipartiteMatching, 65

Blossom, 70

BoundedCirculation, 62

bridge_augmentation, 39

CentroidPairs, 18

cut_tree_values, 58

det_prime, 81

determinant_mod, 82

Dijkstra, 21

Dinic, 53

DirectedEuler, 45

EdgeCompression, 37

EulerLCA, 5

Fenwick, 83

FunctionalGraph, 27

gomory_hu, 57

HLD, 11

Hungarian, 67

LexTwoSAT, 43

LiftingLCA, 8

Lowlink, 40

MatrixTree, 73

MatrixTreeMod, 74

maximum_closure, 56

MinCostFlow, 59

mixed_euler_orientation, 50

mixed_euler_trail, 52

ModInt, 80

odd_cycle_vertices, 35

OfflineLCA, 3

orient_edges, 37

path_intersection, 11

release_bfs, 23

removal_components, 38

SCC, 31

StoerWagner, 63

SubtreeColors, 13

TarjanSCC, 26

TreeDiameter, 1

TwoSAT, 42

UndirectedEuler, 48

VirtualTree, 15

WeightedMatching, 68

word_chain, 47

开放时间最短路, 78

贪吃蛇, 78